指针

2026年10月3日 15:09

一、基本概念

```
#pragma region 知识点一 数据在内存中如何被存储
//位(bit)是计算机中最小的内存单位
//字节(byte)是计算机中最小的存储单位
// 1byte = 8bit

//计算机内存中 每个字节都有一个唯一的内存地址
//无论是32位还是64位的计算机系统，内存地址都是按字节编号的
//这意味着，计算机的内存地址是以字节为单位的
```

```
//什么是内存地址?
//系统会按照 字节 对每个 内存单元 进行编号
//所谓的内存单元，你可以理解为一个个带有门牌号的小房间
//想要使用内存，就需要知道房间的门牌号码
//比如
//声明一个整型变量
// int i = 2;
// int 占4个字节，系统就会为该变量分配4个小房间
// 4个小房间的门牌号码为 5001~5004
```

```
//接着我们再声明一个整型变量
// int j = 10;
// j同样占4个字节，系统会在内存中接着为j分配小房间
// 5005~5008
```



```
//也就说当我们要访问i和j具体存储的信息时
//首先要找到存储i和j对应的房间门牌号码（内存单元们）
// 读取i时，找到i的起始地址5001，因为整型占4个字节
// 因此系统会从5001开始读取4个字节出来作为i的值（5001~5004内存单元）
//
// 同样读取j时，也是一样，j的起始地址为5005，也占4个字节
// 系统会从5005开始读取4个字节作为j的值（5005~5008内存单元）

//而内存单元的门牌号码，我们在程序语言中称之为内存地址（简称地址）
//通过地址（门牌号码）我们就可以访问到指定的内存单元
//所以可以形象的认为 地址 是 指向 内存单元的
```

```
#pragma region 知识点二 指针是什么?
//为了方便记忆，前辈们制定了一个规则
//我们将指向变量地址的对象 形象的称为该变量的 "指针"
//既然 指针 是指向 内存地址 的，那么我们便可以通过指针找到对应的内存单元

//而在程序中
//指针是一个变量，其值为另一个变量的地址
//我们可以通过指针间接的访问和修改该变量的值
//
//因为我们可以通过指针得到它指向的地址（内存地址）
//从而得到对应的内存单元，而内存单元中存储的就是变量的具体值
//所以我们可以通过指针修改变量的值
```

```
//说人话:
//指针变量就是 存储其他变量地址 的变量
//我们不仅可以通过指针得到其他变量的地址
//还可以根据该地址得到其他变量具体的值
```

二、基础使用

```
#pragma region 知识点一 指针的声明
//声明写法:
//变量类型* 指针变量名;
//或
//变量类型 *指针变量名;
//注意:
//1.任何变量类型都可以声明为指针变量类型
//  只需在变量类型后面加一个*
//2.指针变量的常用命名规范
//  前缀法：p或ptr作为前缀
//  后缀法：_p或_ptr作为后缀
//  指针类型变量名：ptr\p+变量类型+变量名 => ptrIntValue、pIntValue
//  等等
#pragma endregion
```

```
#pragma region 知识点二 指针的初始化
//指针必须赋值为内存中的有效地址（已经分配给变量的地址）
//也就是说，根据我们目前学习的知识，我们在使用指针变量时
//必须让他指向一个已经声明了的变量，并且该变量的类型需要和指针变量的变量类型一致
int i = 10;
//如果我们想声明一个指针指向i的内存地址，我们需要获取到i的地址
//我们可以利用 地址符号 &i 来获取
int* p_i2 = &i;
p_i2 = &i;
cout << p_i2 << endl;

//为什么要用相同的变量类型的指针去存储地址
//主要是因为 一个变量它所占用的内存单元可能是多个 而指针变量存储的是它第一个内存单元的门牌号
//如果指针变量类型和存储的变量类型不一致，那么之后就无法准确的读取对应个整个内存单元
//来获取其中的内容
float f = 5.5f;
float* f_ptr2 = &f;
cout << f_ptr2 << endl;
#pragma endregion
```

```
#pragma region 知识点三 指针的解引用操作符
//所谓的 指针解引用操作符 其实就是 *
//如果我们想要获取到指针所指向的内存地址中存储的值
//我们只需要在指针变量前加一个解引用操作符，便可以得到其中存储的值
cout << *p_i2 << endl;
cout << *f_ptr2 << endl;
//并且我们还可以利用它改变原有的值
*p_i2 = 99;
cout << *p_i2 << endl;
cout << i << endl;
#pragma endregion
```

i=99

```
#pragma region 知识点四 空指针和野指针
//空指针是
//不指向任何有效内存的指针
//给指针赋值为 nullptr 或 NULL(老版本) 表示该指针为空指针

//野指针是
//指向无效或不可预知内存地址的指针
//野指针会导致程序出现各种不可预料的错误
//比如：程序崩溃、数据损坏、访问非法内存等等
//主要出现的原因：
//1.未初始化的指针（目前主要遇到的为该种情况）
//2.指向内存被释放后仍使用的指针
//3.指向内存位置超出作用域的指针
//4.指向未定义为分配的内存的指针
//等等

//注意:
//为了避免出问题
//在声明指针时如果不马上使用
//建议将指针声明为空指针
```

七、与字符数组

```
#pragma region 知识回顾1 字符数组
//1.字符数组就是类型为字符char的一维数组

//2.字符数组可以用字符串初始化（默认会在结尾加上\0结束符号）
char charArr[] = "Hellow World";

//3.字符数组可以直接用于打印，但是需要用\0表示结束
char charArr2[] = { 'A', 'B', 'C' };
//会越界，知道遇到\0才会结束，会导致出现乱码
cout << charArr2 << endl;

//遇到\0就会停止
char charArr3[] = { 'A', 'B', 'C', '\0'};
cout << charArr3 << endl;
```

```
#pragma region 知识回顾2 指针和常量
//指向常量的指针：  const在 *前面，值不能改，指向可以改
//指针常量：const在 *后面，值可以改，指向不能改
//指向常量的指针常量：const在 *前后都加，值和指向都不能改了
```

```
#pragma region 知识回顾3 指针和一维数组建立联系
//1.数组变量名
char* ptrChar = charArr;
//2.第一个元素的地址
ptrChar = &charArr[0];
```

```
#pragma region 知识点一 指向常量的字符指针可以直接用字符串赋值
//注意：老版本的C++中甚至是可以不用加const
//指向能变、值不能变
const char* ptrChar2 = "测试用字符串";
```

```
#pragma region 知识点二 字符指针可以直接用于打印
//注意：指针指向的字符数组一定要有\0 否则一样会越界
```

```
#pragma region 知识点三 利用字符指针遍历字符串
//字符指针 用for循环遍历 就不太方便了 因为它有可能指向的是一个字符串字面量 不太好去获取容量
//这种方式一定要保证 有\0 否则会越界
```

```
//为了让字符指针不要自增减去改变指向的位置
//我们可以加一个偏移量 来保证不动
```

```
void printPtrChar( const char* p)
{
    int index = 0;
    while (*(p + index) != '\0')
    {
        cout << *(p + index);
        ++index;
    }
    cout << endl;
}

//总结
//字符指针在很多场景下可以当成字符串或者字符数组来使用
```

八、指针变量作为函数参数

```
#pragma region 知识回顾 按值传递
//当我们在使用函数时
//C++会把实参的副本传递给形参，这意味着函数内部的形参和外部的实参是两个独立的变量
//它们只是初始值相同，对形参的修改只会影响副本，不会改变原始的实参
//形参和实参本质上是两个不同的变量，只是变量中存储的值是相同的
```

```
#pragma region 知识点 指针变量作为函数参数
//当指针变量作为函数参数时
//实参传递给函数体后同样会生成指针变量副本
//但是！！！
//由于副本与原指针指向同一个门牌号
//因此如果修改指针变量指向房间中的内容
//原指针中指向房间中内容也会改变
//也就说，如果你想要在函数内部利用形参改变实参的值
//那么可以利用指针来传递参数，达到目的
```

有一个函数，可以传入一个一维数组
我们传入一个{1,2,3,4,5}，请在该函数内部将其中的3改成99
注意：请在函数中写出三种修改值的方式

有一个函数，可以传入一个二维数组
请声明这个函数，并在该函数中遍历打印传入的二维数组

```
//等等

//注意：
//为了避免出问题
//在声明指针时如果不马上使用
//建议将指针声明为空指针
#pragma endregion
```

```
//总结
//1.指针就是存储变量内存地址的变量
//2.想要获取某一个变量地址 使用 地址符号 &来获取
//3.想要获取指针指向内存地址中存储的值 使用解引用操作符（取值操作符）*
//4.在声明指针时，如果不马上使用该指针，建议大家将其声明为空指针 nullptr
//5.野指针会对程序带来不可预估的问题，需要引起重视
```

请问以下代码是否正确，为什么？
注意：不要通过敲代码验证，直接回答

```
int *p;
int a = 100;
p = 100;
```

请问以下代码的打印结果是多少
注意：不要通过敲代码验证，直接回答

```
int a1 = 10;
int* p1 = &a1;
int a2 = *p1;
a2 = 20;
cout << *p1 << endl;
cout << a1 << endl;
```

请问以下代码的打印结果的含义是什么？
注意：不要通过敲代码验证，直接回答

```
int a = 10;
int* p = &a;
cout << p << endl;
cout << &p << endl;
cout << *p << endl;
```

请利用指针知识比较两个数的大小
提示：指针P1和P2，P1始终要存储更大的值，P2始终要存储最小的值
最后打印信息时利用指针变量打印结果

三、更多使用

```
#pragma region 知识点一 指针变量所占内存空间
//不管何种变量类型的指针变量
//所占的内存空间都是一样的
//因为它是用来存储变量的内存地址的(存储内存单元的门牌号码)
//门牌号码是有范围限制的
//不需要因为变量类型的不同而改变大小
//在不同位数的操作系统 和 不同编译器设置上
//指针所占的字节数有所不同
//32位：4字节
//64位：8字节(常见)
```

```
#pragma region 知识点二 &(地址符号) 和 *(解引用操作符) 的混合使用
//回顾
//& 地址符号 主要用于获取变量地址 用于指针赋值
int* p2 = &a;
cout << p2 << endl;
cout << &p2 << endl;
//* 解引用操作符 主要用于获取地址中存储的内容
cout << *p2 << endl;
```

```
//这两个符号是可以混合使用的
//主要用法有两种
//1.&*
//   使用时从右向左开始结合
//   表示先获取指针指向地址的值，再获取该值的地址
//   相当于获取指针指向的内存地址
int* pp = &*p2;
cout << p2 << endl;
cout << pp << endl;

cout << &p2 << endl;
cout << &pp << endl;
```

p2与pp本身地址不同

```
//2.*&
//   使用时从右向左开始结合
//   表示先获取变量的地址，再获取该地址中存储的值
//   相当于是获取一个变量的值
cout << *&a << endl;
```

打印出的值是a的值

```
//注意：
//只要记住了*和&两个符号的作用
//那么在处理混合使用时，就分开看就可以了，从右往左看即可
//*获取地址中存储的变量的值
//&获取变量的地址的
```

```
#pragma region 知识点三 指针的自增减运算
//指针是可以做自增减运算的
//由于指针存储的是变量的地址
//当我们做指针自增减运算时
//相当于是对地址进行运算
//并且地址的变化是遵循以下规律的
// 自增：将指针向前移动 一个元素的大小
// 自减：将指针向后移动 一个元素的大小
// 元素大小：指针变量的类型决定，sizeof(指针变量类型)得到的就是元素大小
//
// 举例说明：
```

```
#pragma region 知识点四 空类型指针
//空类型指针是一种特殊的指针
//它表示不特定类型的指针
//指针变量可以指向任意的变量类型
//具体存放何种类型，由后续赋值决定
//赋值后，如果需要使用需要强转为对应的数据类型
cout << "***************" << endl;
void* pVoid;
short s = 55;
pVoid = &s;
cout << pVoid << endl;
short s2 = *(short*)pVoid;
cout << s2 << endl;
```

注意不能直接令s2=pVoid 因为不知其变量类型，需强制转换

指针占用多少内存空间？

指针自增减的规则是什么？

请说明以下代码中打印结果的含义

我们传入一个{1,2,3,4,5}，请在该函数内部将其中的3改成99
注意：请在函数中写出三种修改值的方式

有一个函数，可以传入一个二维数组
请声明这个函数，并在该函数中遍历打印传入的二维数组

九、指向函数的指针

```
#pragma region 知识点一 函数在内存中的存储
//我们之前学习了变量的存储
//变量是存储在内存中的小房间中的
//那么函数是否存储在内存中呢？
//答案是肯定的
//只不过它的存储区域有所不同！
//
//当程序启动时，操作系统会将程序的各个部分（包括代码段、数据段、堆和栈）加载到内存中。
//在这个过程中，程序中定义的所有函数的机器码（即执行指令）会被加载到代码段中。
//
//代码段：
//程序内存中的一部分，专门用于存储可执行代码。它是只读的，防止在执行过程中修改代码。
//而每个函数在代码段中都有一个入口地址
//这个地址指向该函数的第一条指令。
//函数名在编译时被映射为这个地址，使得我们可以通过函数名来调用函数！！
```

```
#pragma region 知识点二 指向函数的指针
//指向函数的指针（函数指针）在 C++ 中是一种强大的工具
//它允许你通过指针调用函数、实现回调函数、函数数组等功能
//声明语法：
//返回值类型 (*指针名)(参数类型列表)
//   int (*function_ptr)(int, int)
//   void (*function_ptr)()
//注意事项：
//1.用函数指针指向函数时，函数的返回类型和参数类型必须和函数指针结构上完全一致
//2.函数指针之所以需要用括号把*和指针名括起来，是用于区分 函数指针 和 返回指针的函数
//3.函数指针一般指 指向函数的指针
//   指针函数一般指 返回类型是指针的函数（之后会专门讲）
```

```
//(*func)()和func() 有什么区别？
//本质上没有任何区别，只是写法上不同而已
//func()这样写 编译器在编译的时候其实也会将其转换为 (*func)()
//因此我们平时其实会选择使用更简单的写法 直接函数指针名去调用指向的函数即可
```

十、回调函数

```
#pragma region 知识点一 什么是回调函数
//回调函数(Callback Function)
//是指通过函数指针将一个函数的地址传递给另一个函数
//并在合适的时机调用该函数，它通常用于延迟执行某些逻辑

//说人话：
//回调函数就是 一个函数作为参数传递给另一个函数，并在后者执行过程中调用前者 的一种用法
//先记录某一个函数，过一会儿再调用
//相当于 一会儿回头再调用 的函数
```

```
#pragma region 知识点二 如何使用回调函数
//将函数指针作为某一个函数A的参数，当调用函数A时，将函数作为参数进行传递
//那么这个被传递的函数指针就可以称为回调函数了

//有一个函数指针作为参数的函数，那么传入的这个函数指针 我们一般就称它为回调函数
```

```
#pragma region 知识点三 回调函数对于我们的意义
// 1.延迟执行函数逻辑
// 2.让回调执行的逻辑可以变化！
```

声明一个函数，该函数可以传入两个回调函数（无参无返回）和一个数值，当这个值小于50时回调第一个，大于等于50时回调第二个

```
    int i;
    cin >> i;
    test(i,test1,test2);
}
void test(int i, void(*num1)(), void(*num2)())
{
    if (i<50) num1();

    else num2();
}
void test1()
{
    cout << "执行1" << endl;
}
void test2()
{
    cout << "执行2" << endl;
```

指针自增减的规则是什么?

请说明以下代码中打印结果的含义
注意：不要通过敲代码验证，直接回答

```
int a = 10;
int* p = &a;
cout << *&p << endl;
```

请说明以下代码中可能存在的问题
注意：不要通过敲代码验证，直接回答

```
int a = 70000;
int* p = &a;
cout << *(short*)p << endl;
```

四、与常量

```
#pragma region 常量知识回顾
//常量是指在程序执行过程中其值不能被改变的变量，大部分的常量在声明时都必须被初始化
//关键点:
//1.必须初始化
//2.不能被修改
const int i = 10;
int const i2 = 20;
```

```
#pragma region 知识点一 指向常量的指针
//注意:
//指向常量的指针，并不是只能指向常量，也可以指向普通变量
//这样命名的主要原因是，这种指针不能改变值，类似常量的特点

//指向常量的指针，在*前面加一个const
//特点:
//可以改变指针指向的地址，但是不能改变值（不能直接改变指针指向地址的值）
int j = 1;
const int* p = &j;
//在星号前加上const关键词的指针 称为 指向常量的指针
//不能够改变其中存储的值的
//*p = 10;
//但是可以改变该指针的指向
p = &i;
int const* p2 = &i;
//不能够改变其中存储的值的
//*p2 = 11;
//但是可以改变该指针的指向
p2 = &j;
```

```
#pragma region 知识点二 指针常量
//注意:
//指针常量，不能指向常量，可以指向普通变量
//这样命名的主要原因是，这种指针不能改变指向，但是可以改变指向地址中存储的值

//指针常量，在*后加上const
//特点:
//不能改变指针指向的地址，但是可以改变值（指针指向地址中存储的值）
int j2 = 10;
//指针常量
int* const ptr = &j2;
```

```
//可以改变指针中指向地址中存储的值
*ptr = 20;
cout << j2 << endl;
cout << *ptr << endl;
//但是不能够再改变该指针的指向了
//ptr = &j;

//不能指向常量 因为 它的特点和常量的特点冲突了
int const jj = 99;
int* const ptr2 = &jj;
```

```
#pragma region 知识点三 指向常量的指针常量
//指向常量的指针：可以改变地址指向，不能改变地址中存储的值
//指针常量：不能改变指针指向的地址，可以改变地址中存储的值
//那么，如果想要地址和值都不能改变，那么就可以构建一个指向常量的指针常量
//即*前后都加上const
const int jjj = 10;
int const jjj2 = 20;
//指向常量的指针常量
const int* const ptr3 = &jjj;
//或者这样写
int const* const ptr4 = &jjj2;
```

```
//总结
//指向常量的指针：  const在 *前面去添加即可，值不能改，指向可以改
//指针常量：const在 *后面去添加即可，值可以改，指向不能改
//指向常量的指针常量：const在 *前后都加，值和指向都不能改了
```

五、与一维数组

```
#pragma region 知识点一 指针和一维数组的关系
//数组在内存中是连续存储的
//因此我们只要利用指针获取到数组一开始的位置
//就可以通过指针的自增减获取到数组中的所有信息
```

```
#pragma region 知识点二 建立关系
//关键点:
//1.指针类型需要和数组类型一致
int array[5] = { 1,2,3,4,5 };
int* ptr = nullptr;

//2.获取数组一开始的位置
//方式一：数组变量名即可表示数组的首地址
ptr = array;
cout << ptr << endl;

//方式二：获取数组第一个元素的地址
ptr = &array[0];
cout << ptr << endl;
```

```
#pragma region 知识点三 利用指针获取数组元素
//首地址便是第一个元素
cout << *ptr << endl;

//指针自增减可以移动 元素类型占用内存空间 个单位
cout << *++ptr << endl;
cout << *++ptr << endl;
```

```
}
void test2()
{
    cout << "执行2" << endl;
}
```

十一、空类型指针调用函数

```
#pragma region 知识回顾 空类型指针
//空类型指针是一种特殊的指针
//它表示不特定类型的指针
//指针变量可以指向任意的变量类型
//具体存放何种类型，由后续赋值决定
//赋值后，如果需要使用需要强转为对应的数据类型
```

```
#pragma region 知识点一 空类型指针指向函数
//空类型指针可以指向任意类型的函数
```

```
#pragma region 知识点二 空类型函数指针的使用
//必须将空类型指针强转为 返回值类型，参数类型 符合规范的指针后使用
//注意:
//强转时省略函数指针名即可
```

十二、函数返回值为指针

```
#pragma region 知识点一 返回值为指针的函数是什么
//顾名思义
//既然指针属于变量类型
//那么自然也可以作为函数的返回值
```

```
#pragma region 知识点二 返回指针变量指向地址的生命周期问题
//函数返回的指针返回值，需要分情况分析
//1.局部变量：不允许返回局部变量地址，因为局部变量在函数执行完毕后就会被释放
//           如果在外部使用可能不能得到正确结果，可能还会导致程序崩溃等问题
//           即使打印了正确的值，但也是偶然的，不可靠的，不可预知的
//           因此，不要在返回指针变量时，返回指向局部变量的指针
//2.全局变量：允许返回全局变量地址，因为全局变量生命周期从程序开始，一直持续到程序结束为止
//3.静态变量：允许返回静态变量地址，静态变量初始化后会一直占用内存空间，直到程序结束
```

```
//总结
//函数是指针返回值时，我们返回出去的地址
//不能是局部变量的地址，一定是全局变量或者静态变量的
//因为局部变量用完就会被释放，之后在外部使用可能会出问题
//而全局和静态变量不存在这样的问题，因为他们的生命周期是和程序“同生共死的”
```

编写一个函数，返回指向 两个值相加函数add 或 两个数相减函数multiply 的指针，具体返回哪个函数根据传入的字符（'+' '-'）决定。

声明一个函数，该函数可以传入两个回调函数和一个数值，当这个值小于50时返回第一个回调，大于等于50时返回第二个回调函数。返回后在外部调用

```
int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a - b; }
int (*test(char op))(int, int) // 👈 这种写法很绕，推荐用 typedef
{
    if (op == '+')
    {
        return add;      // 直接返回函数名（函数名即为地址）
    }
    else if (op == '-')
    {
        return multiply; // 返回减法函数
    }
    return nullptr; // 输入非法字符，返回空指针
}
```

```
int main()
{
    int x, y;
    cout << "请选择+或-" << endl;
    char op;
    cin >> op; // 👈 从键盘输入字符
    cout << "输入两个数字" << endl;
    cin >> x >> y;
    // 声明一个函数指针来接住 test 的返回值
```

```
//指针自增减可以移动 元素类型占用内存空间 个单位
cout << *++ptr << endl;
cout << *++ptr << endl;
cout << *++ptr << endl;

cout << *--ptr << endl;
```

```
//还可以直接 + - 但是如果不 += -=的话 自己位置是不会改变的
cout << *(ptr - 1) << endl;
cout << *ptr << endl;

//也可以直接减n个单位
cout << *(ptr -= 2) << endl;
cout << *(ptr += 3) << endl;

//一定注意 不能越界 交
```

```
#pragma region 知识点四 数组名当做指针变量使用
//数组名不能像刚才指针一样 去改变它指向的内存地址
//我们只能利用它的地址来进行计算 不能改变
cout << *array << endl;
cout << *(array + 1) << endl;
cout << *(array + 2) << endl;
cout << *(array + 3) << endl;
```

```
#pragma region 知识点五 利用指针遍历数组
//注意：我们无法直接通过指针获取到数组的容量
//因为指针其实只是记录了一个地址而已，通过地址并不能知道数组容量
int* p = array;
int* p2 = array;
for (int i = 0; i < sizeof(array) / sizeof(int); i++)
{
    //第一种 去偏移计算
    cout << *(p + i) << endl;
    //第二种 去自增
    cout << *p2++ << endl;
    //第三种 把数组名当成指针用 但是不能改变数组名的指向
    cout << *(array + i) << endl;
    //传统方式
    cout << array[i] << endl;
```

数组容量是由数组得到的，而不是由指针

```
//总结
//1.指针变量类型需要和数组保持一致
//2.数组名字和数组首元素的地址 都可以获取到数组开始的位置（首地址）
//3.利用指针的自增自减（加减运算）可以获取到数组中不同的元素，需要注意越界问题
//4.数组名可以用来当作指针使用，但是我们不能去改变数组指向的内容（不能对数组进行重定向）
```

声明一个包含 5 个整数的数组 arr
并使用指针访问数组中的每一个元素，输出它们的值

编写一个程序，使用指针遍历一个整数数组，统计数组中大于10的元素数量，并将这些大于10的元素放置到数组的前部，其余元素保持不变
注意：不能使用额外的数组
提示：使用两个指针（首尾指针）

六、与二维数组

```
#pragma region 知识点一 指针和二维数组的关系
//二维数组在内存中也是连续存储的
//因此我们只要利用指针获取到二维数组一开始的位置
//就可以通过指针的 增减 获取到二维数组中的所有信息
```

```
#pragma region 知识点二 建立关系
//关键点:
//1.指针类型需要和二维数组类型一致
int arr[2][4] = { 1,2,3,4,
                  5,6,7,8 };

//2.按元素建立关系
int* p = &arr[0][0];//得到的是第一行第一列的元素地址
int* ptr = arr[0];//得到的是第一行的地址 = 第一行第一列的元素地址
```

```
//3.按行数组建立关系
//利用数组指针，获取二维数组中一行的一维数组起始地址
//数组指针是一个指向整个数组的指针
//本质上是一个指针，指向一个包含n个元素的数组
//数组指针定义规则：变量类型 (*指针名)[元素个数]
//
//int (*p2)[4] 表示指向包含4个int元素的一维数组的指针
//int：数组中元素的类型
//*p：指针变量
//[4]：表示这个指针指向的数组包含4个int元素
//指向二维数组中第一个一维数组的指针
int (*p2)[4] = arr;
```

```
#pragma region 知识点三 利用指针获取二维数组元素
//由于二维数组的存储序列也是连续的
//因此利用指针增减也可以获取到所有元素
//1.按元素获取的指针
cout << "按元素获取内容" << endl;
cout << *p << endl;//1
cout << *(p + 1) << endl;//2
cout << *++p << endl;//2
cout << *--p << endl;//1
cout << *(p += 3) << endl;//4
cout << *++p << endl;//5
cout << *(p + 3) << endl;//8
//注意:
//要在范围内访问0<= i < 行*列 之间
//不要越界，不要越界，不要越界
```

```
//2.用数组指针获取元素
// p2是什么?
// 类型是：数组指针 int (*)[4]
// 指向的内容：指向二维数组中的第一个一维数组
// 每次增减 偏移位置为：元素字节数 * 容量
cout << p2 << endl;
cout << p2 + 1 << endl;//偏移元素*容量 16个字节
// *p2是什么?
// 类型是：指向一维数组指针 int*
// 指向的内容：指向二维数组中的第一个一维数组中的第一个元素
// 每次增减 偏移位置为：元素字节数
cout << *p2 << endl;
cout << *p2 + 1 << endl;//偏移元素的 4个字节
```

```
#pragma region 知识点四 数组当做指针变量使用
//1.利用行地址来获取元素
//  先得到行首地址，在通过加减来获取某行某元素

//2.利用二维数组名来获取元素
//关键知识:
```

```
    cout << "输入两个数字" << endl;
    cin >> x >> y;
    // 声明一个函数指针来接住 test 的返回值
    int (*p)(int, int) = test(op);
```

```
    if (p != nullptr)
    {
        // 通过函数指针调用函数
        int result = p(x, y);
        cout << "x 和 y 的运算结果是: " << result << endl;
    }
    else
    {
        cout << "输入错误! " << endl;
    }
    return 0;
}
```

十三、指针数组

```
#pragma region 知识回顾 数组指针
//数组指针是我们在学习指针和数组中 二维数组相关知识时学习的
//数组指针是一个指向整个数组的指针，本质上是一个指针，指向一个包含n个元素的数组

//数组指针定义规则：变量类型 (*指针名)[元素个数]
//int (*p2)[4] 表示指向包含4个int元素的一维数组的指针
int arr[3][4] = { 1,2,3,4,
                  5,6,7,8,
                  9,10,11,12 };
int (*p)[4] = arr;
cout << p << endl;
cout << *p << endl;
cout << (*p)[1] << endl;
cout << *((*p) + 1) << endl;
```

```
#pragma region 知识点二 指针数组的使用
//声明
int b = 11;
int c = 12;
int* p3[] = { &a, &b, &c };
//查找地址
cout << p3[0] << endl;
cout << p3[1] << endl;
cout << p3[2] << endl;
//查找地址对应的值
cout << *p3[0] << endl;
cout << *p3[1] << endl;
cout << *p3[2] << endl;
//修改指向地址
int d = 13;
p3[0] = &d;
cout << p3[0] << endl;
cout << *p3[0] << endl;
```

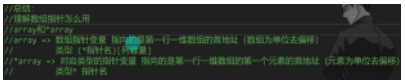
```
//修改值
*p3[0] = 99;
cout << p3[0] << endl;
cout << *p3[0] << endl;
```

```
//1.利用行地址来获取元素
//  先得到行首地址，在通过加减来获取某行某元素

//2.利用二维数组名来获取元素
//关键知识：
// 二维数组名 可以理解为一个 int (*)[4] 类型的指针
//arr和*arr的区别
//就像前面讲解的 p2 和 *p2 的区别
```

```
//二维数组名和数组指针有什么区别？
//他们通过sizeof获取容量时
//二维数组名得到的是实实在在的这个二维数组占用了多少个字节
//数组指针（只要是指针）占用的内存空间是8个字节
cout << sizeof(p2) << endl;
cout << sizeof(arr) << endl;
```

知识点五 利用数组指针遍历二维数组



有一个二维数组array
请说明array、array[0]、&array[0][0]的作用和区别

有一个二维数组array
请说明
&array[n][m]、
array[n] + m、
*(array + n) + m、
代表什么？
以及
array[n][m]、
*(array[n] + m)、
((array + n) + m)
又代表什么

声明一个 2x3 的二维数组 arr 并初始化为 {{1, 2, 3}, {4, 5, 6}}。编写代码输出每个元素的值，要求使用指针访问二维数组中的每个元素（禁止使用下标运算符 []）

```
*p3[0] = 99;
cout << p3[0] << endl;
cout << *p3[0] << endl;
cout << d << endl;

//遍历
//容量
//得到指针数组一共占多少字节
cout << "得到指针数组一共占多少字节:" << sizeof(p3) << endl;
//得到指针中一个元素占多少字节
cout << "得到指针中一个元素占多少字节:" << sizeof(p3[0]) << endl;
//得到容量
cout << "得到容量:" << sizeof(p3) / sizeof(p3[0]) << endl;

for (int i = 0; i < sizeof(p3) / sizeof(p3[0]); i++)
{
    cout << p3[i] << endl;
    cout << *p3[i] << endl;
}
```

弄清楚地址关系即可

```
for (int i = 0; i < sizeof(p3) / sizeof(p3[0]); i++)
{
    //指针数组中第i个元素的地址
    cout << p3 + i << endl;
    //指针数组中第i个元素中存储的值（因为是指针数组 所以取出来的也是地址）
    cout << *(p3 + i) << endl;
    //指针数组中第i个元素中存储的地址指向的值
    cout << **(p3 + i) << endl;
}
```

```
//总结
//对于指针数组来说
//最重要的就是理解指针数组在内存中存储信息的关系
//主要理解值和地址的关系
//指针数组名和本身地址的关系即可
```

编写一个程序，完成以下任务：
1.定义三个函数，分别实现以下功能：
  add(int a, int b)：返回 a 和 b 的和。
  subtract(int a, int b)：返回 a 和 b 的差。
  multiply(int a, int b)：返回 a 和 b 的积。
2.创建一个指针数组，存储指向这些函数的指针。
3.通过指针数组调用每个函数，并打印结果。

编写一个程序，完成以下任务：
1.创建一个 3x3 的二维整型数组，并初始化如下：
  1 2 3
  4 5 6
  7 8 9
2.创建一个指针数组，让每个指针指向二维数组的一行。
3.使用指针数组，输出二维数组中的所有元素。

十四、多级指针

```
#pragma region 知识点一 多级指针是什么
//多级指针就是指向指针的指针
//可以理解为一种指针的嵌套结构
//是用来存储指针变量的地址的
```

```
#pragma region 知识点二 一级指针
//我们目前常用的指针都是一级指针，主要用来存储普通变量的地址
//举例
int a = 99;
int* p = &a;
//变量的地址
cout << p << endl;
//变量的地址存储的值
cout << *p << endl;
```

```
#pragma region 知识点三 二级指针
//二级指针是用来指向 一级指针变量地址 的指针
//举例
int** pp = &p;
//指针p的地址
cout << pp << endl;
//指针p的地址中存储的内容（由于指针p是指针，
cout << *pp << endl;
//指针p的地址中存储的地址中对应的值
cout << **pp << endl;
#pragma endregion
```

三级指针同理

```
#pragma region 知识点五 多级指针
//多级指针的本质就是用来存储 指针变量的地址 的
//多级指针可以有n级，第 n级 指针是用来存储 n-1 级指针的地址的
//如果想要获取多级指针最底层的值，只需要再前面加n个*即可
```

编写一个函数，在函数内部需要将传入的指针改变指向到一个函数内部的静态变量，该修改会影响实参的指向

内存分配

2026年10月3日 15:10

一、内存存储区域

```
#pragma region 知识点一 不同的内存存储区域
//在程序运行时，系统中会有不同的内存存储区域
//
//我们目前学习过的知识，涉及的存储区域有：
//1.代码段（Code Segment，或称代码存储区）：
//  存储 函数的机器码（执行指令）等信息的区域
//  通常是只读的，防止程序意外修改自己的代码
//  在程序加载时就被加载到内存中
//  生命周期较长，与程序同生共死
//
//2.数据段（Data Segment，或称数据存储区）：存储 全局变量、静态变量 等信息的区域
//  一般分成两个子区域
//  初始化数据段：存储已初始化的全局和静态变量
//  未初始化数据段（BSS Segment）：存储未初始化的全局和静态变量，通常初始化为零
//  生命周期较长，与程序同生共死
```





```
//3.常量段（Constant Segment，或称常量存储区）：存储 字符串字面量和其他只读数据 信息的区域
//  通常是只读的，以防止程序修改常量的值
//  比如 cout << "Hello, World" << endl 中的 "Hello, World" 就存储在常量区
//  生命周期由程序控制
//  注意：我们声明的常量变量不存储在常量段，而是根据在哪里声明决定存储在哪里
```



```
//4.栈（Stack）：存储 局部变量、函数参数、返回值 等信息的区域
//  由编译器自动管理，随着函数的调用和返回而分配和释放
//  生命周期与函数调用相同，快速分配和释放
//
//未学习的存储区域：
//5.堆（Heap）：存储 动态分配的内存数据 等信息的区域
//  由程序员手动管理，需要主动分配，主动释放
//  生命周期不固定，由程序员决定生死
```

```
#pragma region 知识点二 不同存储区域对于我们的影响
//由于 全局变量、静态变量、函数等内容
//在生命周期的角度上一般不会出现什么问题（不会出现被销毁了找不到的情况）
//
//而会因为生命周期问题给我们带来麻烦的主要是栈和堆上存储的内容
//因此在整个开发生涯中我们需要明确区分存储在栈和堆上的内容

//栈（Stack）
//由编译器自动管理，我们无需手动管理
//堆（Heap）
//由程序员手动管理，需要主动分配，主动释放
```

```
#pragma region 知识点三 感受栈
//函数中的局部变量体现
//我们之前学习指针函数（返回值为指针的函数）充分表现了这一点
int* add(int a, int b)
{
    int c = a + b;
```

```
#pragma region 知识点三 感受栈
//函数中的局部变量体现
//我们之前学习指针函数（返回值为指针的函数）充分表现了这一点
int* add(int a, int b)
{
    int c = a + b;
    int* p = &c;
    return p;
}
```

二、堆上分配内存

```
#pragma region 知识回顾 堆是什么
//存储 动态分配的内存数据 等信息的区域
//由程序员手动管理，需要主动分配，主动释放
//生命周期不固定，由程序员决定生死
```

```
#pragma region 知识点一 如何在堆上分配内存
//使用new关键字配合指针使用 即可在堆上分配内存
//1.单个对象分配
//语法:
//变量类型* 指针名 = new 变量类型(初始值（可选）)
//作用:
//new 的作用就是“开房”（分配内存房间，门牌号，和具体的内存容量）
//注意:
//指针变量本质上还是在栈上，只是其中存储的地址是堆上的地址
//指针变量本身还是由系统管理，我们需要手动管理的是他指向的地址
int* p = new int(10);
```

数值类型默认初始值是0

```
//2.分配数组
//语法:
//数组类型* 指针名 = new 数组类型[容量] {初始值（可选）};
//作用:
//new的作用同样是“开房”（分配了n个连续的内存房间，具体是否初始化房间里的值，自己定）
//这时 指针arr相当于就是数组名
int* arr = new int[4] {1,2,3,4};
cout << arr[0] << endl;
cout << arr[1] << endl;

int (*arr2)[3] = new int[2][3]{ 1,2,3,4,5,6 };
```



```
#pragma region 知识点二 如何在堆上释放内存
//使用delete关键字 即可在堆上释放内存
//语法:
//delete 指针变量 —— 配对 new
//delete[] 数组变量 —— 配对 new[]
//作用:
//delete的作用就是释放通过new在堆上分配的动态内存
//注意:
//delete的作用并不是销毁指针变量本身，而是销毁指针变量指向的堆内存
```

```
//1.释放单个对象
delete p;
p = nullptr;
```

释放后令指针指向空，否则可能出现野指针

```
//2.释放动态分配的数组
delete[] arr;
arr = nullptr;
```

```
//2.释放动态分配的数组
//注意：释放数组时一定要加上[]，否则只会释放第一个元素，会造成内存泄漏或崩溃
delete[] arr;
arr = nullptr;
```

请问这句代码是在做什么？是否存在问题
int i = (int)(new int(100));

有一个在堆上的分配的数组，容量为5，其中存储了{1,2,3,4,5}
现在希望将数组容量扩容到10，并且保留原数组中的内容，请实现该功能

请实现一个返回指针的函数，用于返回两个数相加的和。

三、内存安全

```
#pragma region 知识点一 什么是内存安全
//内存安全（Memory Safety）指的是程序在访问和操作内存时不会因为出现程序问题
//导致程序报错、崩溃的情况。
//想要保证内存安全，我们主要保证以下几点即可
//1.了解内存销毁时机
//2.防止内存泄漏
//3.防止越界访问
//4.空指针检查
//5.避免使用野指针
```

```
#pragma region 知识点二 了解内存销毁时机
//1.栈内存
//   在函数调用结束时，局部变量的内存会自动销毁。
//   栈内存是自动管理的，一旦程序退出某个作用域（例如函数或代码块）
//   该作用域内的局部变量的内存会自动释放
//   因此我们只需注意不要在作用域外使用作用域内的在栈上分配的内存
//
//2.堆内存
//   使用 new 分配的堆内存需要在不使用后通过 delete 或 delete[] 进行释放
//   因此我们只需要注意控制堆内存对象的生命周期，不用时一定要删除
```

```
//堆和栈的其他区别：
//1.内存空间大小不同
//   栈的大小通常是固定的（比如几兆），是由操作系统决定的。
//   因此我们不能在其中分配大内存
//   堆的大小取决于系统可用的内存，理论上可以分配很大的内存
//   因此我们可以在其中分配大内存
```

```
//2.速度不同
//   栈的分配和释放速度快，因为是由系统直接管理
//   堆的分配和释放速度较慢

//3.线程安全
//   栈是线程私有的，不同线程的栈彼此独立，是线程安全的
//   堆是进程共享的，多线程访问时需要保证线程安全

//如何应用
//栈：适用于小规模，短生命周期的数据
//堆：适用于动态分配的大规模数据，或生命周期跨越函数调用的数据
```

```
#pragma region 知识点三 防止内存泄漏
//内存泄漏主要指不再被使用的内容一直占用内存资源
//在使用堆内存时很容易造成这样的结果
//1.堆内存忘记释放（一直占用堆内存）
//2.丢失对已分配堆内存的记录（无法释放，一直占用堆内存）
//出现以上类似的情况就会造成内存泄漏
//当内存达到一定的瓶颈是就会崩溃或报错，程序无法正常运行
```

```
#pragma region 知识点四 防止越界访问
//我们需要限制数组或指针的访问范围
//要避免访问超出分配范围的内存
```

```
#pragma region 知识点五 空指针检查
//养成好习惯
// 1.在声明指针没有初始化时对它赋值为nullptr
// 2.对于指向堆内存的指针，释放后，同样对它赋值为nullptr
//这样在使用指针之前判断它是否为nullptr空指针再决定如何使用它
int* p = nullptr;
p = new int(10);
delete p;
p = nullptr;

if (p != nullptr)
{

}
```

```
#pragma region 知识点六 避免使用野指针
//避免指针指向已释放的内存空间
//确定指针指向的内存不能用时，一定置空
```

引用

2026年10月4日 22:47

一、基础使用

https://www.bilibili.com/video/BV1kMqbYDEqP/?spm_id_from=333.788.videopod.episodes&vd_source=747b172eede8b83370ffc0953b28b100&p=35

```
#pragma region 知识点一 什么是引用
//C++中的引用概念一般指通过别名来操作变量
//我们可以利用 & 符号来声明引用变量
//我们之前学习的&是作为地址符号来使用的，主要用在指针赋值时获得变量地址的
//而&符号的另一个作用就是用来声明引用

//关键点:
//引用是已有变量的别名

//基础语法
//变量类型& 引用名 = 表达式;
```

```
#pragma region 知识点二 使用引用的注意事项
//1.引用对象必须初始化

//2.引用变量的类型必须和引用的内容类型一致

//3.引用绑定的对象不能更改

//这里并不会修改引用的绑定，只会进行赋值

//4.引用不会占用新的内存空间，只是原变量的另一种名称而已
```

避免悬空引用

```
#pragma region 知识点三 引用和指针的区别
//1.引用必须被初始化并且不可以修改指向，而指针可以为空并随时可以切换指向
int* p2 = nullptr;
p2 = &a;
p2 = &b;
cout << *p2 << endl;
//2.引用更安全，不存在空引用问题，并且自动解引用不需要加*获取值，语法上更清晰

//3.指针可以进行加减运算移动指向位置，引用不行
```

二、左值引用和右值引用

https://www.bilibili.com/video/BV1kMqbYDEqP/?spm_id_from=333.788.videopod.episodes&vd_source=747b172eede8b83370ffc0953b28b100&p=36

```
#pragma region 知识点一 左值和右值的含义
//左值和右值是表达式结果的分类
//代表 表达式在计算时的特性，主要用于区分持久性和是否具有存储位置

//左值:
//指在表达式结束后仍然存在的对象，通常是可以被赋值的，一般在一定周期内时持久存在的
//特点:
//1.左值有明确的存储位置（可以获取到地址）
//2.可以出现在赋值符号的左边，允许被赋值修改
//举例:
int x = 5;//x就是左值，具有存储位置
x = 10; //可以给左值x赋值
int* p = &x;
```



```
//右值:
//指临时对象或不具持久性的值，一般在表达式计算完成后即被释放
//特点:
//1.右值没有明确的存储位置（一般不能获取地址）
//2.右值表达式一般只能出现在赋值符号=的右边
//3.右值一般是临时计算的结果，或者没有名称的值，比如字面值、临时对象等
//举例:
int y = 5 + 10;//这里赋值符号右侧的结果15就是一个右值，不能被赋值的 15 = 多少
//int* p2 = &(5 + 10);//对于右值来说是无法获取到地址的 会报错

//简单的去记忆左右值:
//左值就是有名称的，可以被持续引用的对象
//右值就是一些在计算中产生但不会长期存储的表达式结果，是临时的值，用完即弃的值
```

```
#pragma region 知识点二 左值引用
//我们上节课学习的引用的基础使用中使用的引用就是左值引用
//使用 引用符号& 来进行引用变量的声明
int a = 10;
int& refA = a;
//左值引用只能绑定到左值（即在表达式结束后仍存在的变量）
//主要用于指向持久存储在内存中的变量或对象
//没有办法去指向右值
```

```
#pragma region 知识点三 右值引用
//右值引用是使用 两个 && 声明的引用对象
//它只能绑定到右值（即临时的、没有名称的对象，比如函数返回值、字面量等）
//特点:
//1.可以指向其它右值
//2.右值引用延长了右值的生命周期
//注意:
//我们目前可能不太感受得到右值引用的意义
//随着我们的学习，以后我们会渐渐明白的
```

三、引用和函数

https://www.bilibili.com/video/BV1kMqbYDEqP/?spm_id_from=333.788.videopod.episodes&vd_source=747b172eede8b83370ffc0953b28b100&p=37

```
#pragma region 知识点一 使用引用传递函数参数
//左值引用参数
//用于将函数形参绑定到可修改的左值变量，避免值拷贝并且允许函数内修改传入的对象
//它的作用类似使用指针传递函数参数，但是形参不会重新创建副本了，而是直接和实参绑定，是实参的别名
//但是还是必须遵守引用使用的规则：
//1.引用变量的类型必须和引用的内容类型一致
//2.引用绑定的对象不能更改
//3.引用不会占用新的内存空间，只是原变量的另一种名称而已
//4.通过引用传递的参数，当函数执行完毕后并不会释放该对象，具体何时释放取决于外部的情况
```



```
//右值引用参数
//右值引用的主要目的是优化性能
//避免临时对象或大对象的拷贝，可以直接“拿走”这些对象的资源
//
//但是结合我们目前学习的知识还不太能体现出来
//随着知识的学习，以后用到它时再来体会它的作用
```

```
#pragma region 知识点二 常量引用参数
//可以在函数中声明引用参数时，在前面加上const关键词
//让其变成一个常量引用参数
//通常用于只读操作，确保函数内只会用不会改
//注意：只有左值引用参数才能加const关键字
```

```
#pragma region 知识点三 函数返回引用
//函数是可以返回引用的
//但是要注意不要返回局部变量的引用
//我们目前也会用的较少，随着知识的学习，会用的更多
//使用返回引用的函数时 需要注意的就是生命周期相关 一定要合法合规
```

请为一维数组、二维数组、指针
声明一个引用别名

编写一个函数，接收一个 int 类型的左值引用作为参数，将其值加1。调用该函数后，验证原变量的值是否被修改

编写一个函数，返回一个数组中某个元素的左值引用，并通过该引用修改数组中元素的值

编写一个函数，使用 const 左值引用接收参数并打印参数值，确保函数能接受左值和右值

```
int main()
{
    int arr1[5] = { 1,2,3,4,5 };
    int arr2[3][2] = { 6,7,8,9,10,11 };
    int* p = arr1;
    int (&r1)[] = arr1;
    cout << r1[1] << endl;
    int (&r2)[][2] = arr2;
    cout << r2[1][1] << endl;
    int* (&r3) = p;
    cout << r3[1] << endl;
    return 0;
}
```

枚举

2026年10月5日 17:22

一、枚举

```
#pragma region 知识点一 基本概念

#pragma region 1.枚举是什么
//枚举是一个比较特别的存在
//它是一个被命名的整形常量的集合
//是一种用户自定义的数据类型
//一般用它来表示 状态 类型 等等
```

```
#pragma region 2.声明枚举 和 声明枚举变量
//注意：声明枚举 和 声明枚举变量 是两个概念
//声明枚举：       相当于是 创建一个自定义的枚举类型
//声明枚举变量：   使用声明的自定义枚举类型 创建一个枚举变量
```

```
#pragma region 3.声明枚举语法
enum E_枚举名字
{
    枚举项名字, //0 如果不自己赋值 默认第一项 就会是0
    枚举项名字1,//1
    枚举项名字2,//2
    枚举项名字3,//3
    //......n个枚举项都可以 根据自己的需求添加
};
```

```
enum E_EnumName
{
    Name = 100,//虽然枚举中的值 允许手动改为一样的值 但是不建议
    Name1 = 100,
    Name2,//101
    Name3,//102
    Name4 = 200,
    Name5,//201
    Name6,//202
};
```

```
//枚举变量的声明可以写在任意位置
//注意：声明枚举变量时要保证枚举在之前进行了声明
```

```
#pragma region 知识点四 枚举和整数之间的转换
//枚举转整型
//可以隐式转换为整数类型

//整型转枚举
//需要使用 static_cast<枚举类型>(整型值) 方法进行强制转换
```

```
#pragma region 知识点五 枚举的作用
//在游戏开发中，对象很多时候 会有许多的状态
//比如玩家 有一个动作状态 我们需要用一个变量或者标识 来标识当前玩家处于的是哪一种状态
//综合考虑 可能会使用 int 来标识它的状态
// 1 行走
// 2 待机
// 3 跑步
// 4 跳跃
// .......等等
```

```
//这个时候 就可以通过枚举来帮助我们 清晰的分清楚状态的含义 以及进行逻辑处理
```

定义QQ状态的枚举，并提示用户选择一个在线状态，我们接受输入的数字，并将其转换成枚举类型

用户去星巴克买咖啡，分为中杯（35元），大杯（40元），超大杯（43元），请用户选择要购买的类型，用户选择后，打印：您购买了xxx咖啡，花费了xx元
例如：你购买了中杯咖啡，花费了35元

请用户选择英雄性别与英雄职业，最后打印英雄的基本属性（攻击力，防御力，技能）
性别：
男（攻击力+50，防御力+100）
女（攻击力+150，防御力+20）
职业：
战士（攻击力+20，防御力+100，技能：冲锋）
猎人（攻击力+120，防御力+30，技能：假死）
法师（攻击力+200，防御力+10，技能：奥术冲击）

举例打印：你选择了“女性法师”，攻击力：350，防御力：30，职业技能：奥术冲击

二、强类型枚举

```
#pragma region 知识点一 枚举可能存在的问题
//1.命名冲突
//   多个枚举中存在同名枚举项，会引发命名冲突
//   会和全局变量产生冲突
```

```
//2.隐式转换为int，不够安全
//   可能会不小心和整数进行混合运算
//3.不同枚举类型之间可以产生比较
//   不太复合逻辑
//4.默认枚举项类型只能是int，大材小用了
```

```
#pragma region 知识点二 强类型枚举
//强类型枚举是用来提升枚举的安全性
//语法：
// enum class 枚举名
// {
//     各枚举项...
// };
//进行枚举变量赋值时
// 枚举类型 变量 = 枚举类型::枚举项
```

```
//2.强类型枚举无法隐式转换为整型，需要使用 static_cast<int>(枚举值) 进行显示转换
//   可以避免不小心将枚举和整数混用

//3.可以禁止不同类型的枚举之间进行比较

//4.可以指定枚举的底层类型
//   默认枚举中存储的枚举项是int类型
//   我们可以利用强类型枚举指定其存储类型
//   但是必须是整型类型，不能是其他类型
```

结构体

2026年10月5日 18:04

一、基本用法

```
#pragma region 知识点一 基本概念
//结构体 是一种自定义变量类型  类似枚举需要自己定义
//它是数据和函数的集合
//在结构体中 可以声明各种变量和方法

//作用：用来表现存在关系的数据集合 比如用结构体表现 学生 动物 人类 等等
```

```
#pragma region 知识点二 基本语法
//结构体关键字 struct

//结构体声明
//注意：结构体名字命名时 我们一般用帕斯卡命名法
//struct 自定义结构体名字
//{
//    //第一部分
//    //变量
//
//    //第二部分
//    //构造函数和析构函数(可选)
//
//    //第三部分
//    //函数（方法）
//};
//举例说明
struct Student
{
    //变量 表示学生有的一些属性
    //变量类型 可以写任意的类型 包括结构体（但是 不能是自己的结构体变量-指针和引用除外
    int age;
    bool sex;
    int number;
    string name;

    Student* s;//可以声明自己类型的结构体变量 但一定是指针或者引用
    Student& s2;

    //函数（方法）表示它的一些行为
    void speak()
    {
        cout << "我的名字是" << name << "我今年" << age << "岁" << endl;
    }
};
```

```
#pragma region 知识点三 声明对象语法
//方式一：聚合初始化（数组初始化就是聚合初始化）
//在声明结构体对象后直接通过{}按变量顺序进行初始化
//聚合初始化允许通过大括号 {} 的方式直接对聚合类型的成员变量逐个赋值

//方式二：直接声明
//定义结构体类型的同时声明结构体变量
//直接在结构体类型声明的大括号后声明变量

//方式三：匿名声明
//省略结构体类型名
//直接声明结构体类型变量

//方式四：堆上声明
//可以在堆上分配结构体对象
//一定记住销毁释放
```

```
#pragma region 知识点四 成员的使用
//1.普通变量
//  直接点(.)
//2.指针变量
//  直接指向(->)
```

使用结构体描述小怪兽
小怪兽有攻击力、防御力、闪避率、暴击率等等属性

使用结构体描述学员的信息，姓名，性别，年龄，班级，专业，创建两个学员对象，并对其基本信息进行初始化并打印

二、结构体嵌套、大小、内存对齐

```
#pragma region 知识点一 结构体的嵌套使用
//1.我们可以在一个结构体中包含结构体 甚至声明结构体

//2.内部嵌套的结构体，我们也可以在外部使用它来声明对象
//  语法：外层结构体名::内层结构体名::内层结构体名.....(支持多层嵌套)
```

```
#pragma region 知识点二 结构体的内存占用大小
//使用 sizeof 可以有效地获取结构体的大小，包括成员变量和填充字节（为了满足特定的字节对齐要求）


//什么是字节对齐（内存对齐）
// 在计算机内存中，数据结构的起始地址需要满足特定的规则
// 以提高内存访问的效率
// 字节对齐确保数据在内存中的存储位置符合其类型的对齐要求
// 从而避免因不对齐访问而导致的性能损失或潜在的硬件异常
//
// 每种数据类型通常都有一个对齐要求，即它在内存中存储时必须位于某个特定的字节边界上
// 比如：
//      int可能要求4字节对齐
//      double可能要求8字节对齐
//      char通常要求1字节对齐
```

```
// 为了满足对齐要求，编译器会在结构体成员之间插入填充字节。
// 这些填充字节不会被我们直接访问，但它们确保每个成员的地址都是对齐的
// 合理的字节对齐可以提高 CPU 访问内存的效率。
// 现代 CPU 通常会在特定的字节边界上进行读取和写入操作，对齐访问通常比非对齐访问快得多
//
//对于结构体大小而言
//它的字节占用大小通常是其最大成员对齐要求的倍数
//编译器可能会在成员之间插入填充字节，以满足对齐要求
//填充字节通常会放在结构体的成员之间或结构体的末尾
```

四、析构函数

```
#pragma region 知识点一 析构函数的基本概念
//析构函数 是当对象被释放时会调用的方法
//也就意味着在析构函数中主要处理的就是堆内存回收相关的工作
//因为堆内存空间需要我们自己手动回收

//它的主要作用就是用来当结构体对象被释放时
//能够将其中在堆上分配的内容清理掉，避免内存泄漏
```

```
ragma region 知识点二 析构函数的声明
基本规则
声明在结构体中
1.没有返回值,没有参数
2.函数名必须和结构体名相同，并且在前面加~符号
3.只能有一个析构函数，不能重载
4.如果类中没有定义析构函数，编译器会自动生成一个默认的析构函数，该函数会依次调用所有成员的析构函数
```

```
#pragma region 知识点三 析构函数的使用
//析构函数不需要我们手动调用
//它是自动调用的函数
//当对象的生命周期结束时，析构函数会自动被调用
//对象释放时机：
//1.栈中的内容，当对象超出作用域时就会释放
//2.堆中的内容，delete 删除对象时就会释放
```

```
//总结
//析构函数 不需要我们自己手动调用
//他会在对象生命周期结束时自动调用
//我们只需要把重点放在 在析构函数中 去释放结构体中在堆上分配的内容即可
```

五、结构体和函数

```
#pragma region 知识点一 结构体作为函数参数
//当结构体作为参数传递时
//可以通过以下三种方式：
//1.按值传递（拷贝副本传递给函数）
//2.指针传递（传递结构体地址）
//3.引用传递（传递原结构体）
//对于小型结构体三种方式随便用
//对于结构复杂的建议使用引用或指针传递
```

```
#pragma region 知识点二 函数返回结构体
//结构体和其他变量类型一样，可以作为返回值返回
//注意事项也是一致的，要考虑返回对象的生命周期

//函数返回值的时候
//1.不能去返回局部变量（在函数内部声明的变量）生命周期会随着函数调用的结束而结束
//  如果在外部继续去使用这种局部变量 会出现问题
//2.除非你这个局部变量时在堆上去分配的空间，那么可以直接返回，因为堆上内容需要我们自己管理释放
//3.否则就只能返回 静态变量 或者 全局变量 才可以，或者是外部传入的指针或者引用变量才行

//总结，不管变量类型是什么 只要要作为函数返回值，那我们就必须考虑它的生命周期问题
```

应用已学过的知识，实现奥特曼打小怪兽
提示：
结构体描述奥特曼与小怪兽
定义一个方法实现奥特曼攻击小怪兽
定义一个方法实现小怪兽攻击奥特曼

六、结构体数组

```
struct Student
{
    int age;
    string name;
}students[5];

int main()
{
    std::cout << "结构体数组\n";

    #pragma region 知识点一 结构体数组的声明和初始化
    //和普通结构体声明一致，只不过声明为数组即可
    cout << students[0].age << endl;

    Student arr[5] = { {10, "111"},{11, "112"} ,{12, "113"} ,{13, "114"},{14, "115"} };
```

```
#pragma region 知识点二 指针和结构体数组
//规则和指针和一维数组规则一致
Student* p = arr;
cout << p->age << endl;
cout << p->name << endl;
p++;
cout << p->age << endl;
cout << p->name << endl;
```

定义一个数组存储10个上节课练习题中描述的小怪兽，每个小怪兽的名字为（小怪兽+数组下标）
举例：小怪兽0，最后打印10个小怪兽的名字+攻击力数值

```
//规则：
//1.每个成员变量的起始地址必须是其类型大小的倍数
//2.整个结构体的大小是最大成员对齐要求的倍数
//3.结构体嵌套时，外部结构体需要考虑内部结构体成员变量的字节最大数
```

```
#pragma region 知识点三 内存对齐带来的问题
//内存对齐的目的是提高内存的访问性能
//但是会带来浪费内存的问题
//而且通过举例我们发现，结构体中变量类型的不同顺序会带来不同的内存占用大小
//因此我们应该按照以下的规则去排列变量
//1.按照大小降序排列成员，占用越大空间的变量类型放在前面
//2.减少不必要的成员变量
//3.使用更小的数据类型
//等等（有些手段以后再学）
```

三、构造函数

```
#pragma region 知识点一 构造函数的基本概念
//基本概念：
//构造函数是一个特殊的成员函数
//是用来对结构体对象进行初始化用的
//当我们声明一个结构体变量（对象）时，会自动调用构造函数
struct Test
{
    //成员变量

    //成员函数（方法）

    //构造函数 析构函数
};
```

```
#pragma region 知识点二 构造函数的声明
//基本规则
//声明在结构体中
//1.没有返回值
//2.函数名必须和结构体名相同
//3.可以被重载
//4.两种赋值方式
//  4-1:在函数中赋值
//  4-2:利用成员初始化列表来初始化（更高效）
//5.若传入参数和成员变量重名，可以使用 this->成员变量 来表示是成员变量
```

```
#pragma region 知识点三 构造函数的使用
//调用构造函数的几种方式
//1.栈上分配-不带new
//  1-1.结构体类型 结构体变量(参数...)
//  1-2.结构体类型 结构体变量{参数...}
//  1-3.结构体类型 结构体变量 = 结构体类型(参数...);
//  1-4.结构体类型 结构体变量 = 结构体类型{参数...};
//  1-5.结构体类型 结构体变量 = (参数...);
//  1-6.结构体类型 结构体变量 = {参数...};
//2.堆上分配-带new
//  结构体类型 结构体变量 = new 结构体类型(参数);
```

1-5不建议使用

```
//结构体还有两个默认的构造函数
//1.传入 const 左值引用的构造
//  主要作用：避免不必要的复制，允许使用已有的对象用来初始化成员
//  一般用来处理复制另一个对象的数据
//2.传入 右值引用的构造
//  主要作用：接收将要被移动的临时对象或其他右值，可以提高性能
```

```
#pragma region 知识点四 显示定义默认构造函数
//如果我们没有为结构体定义任何构造函数
//编译器会生成一个默认的构造函数
//如果定义了其他构造函数（如带参数的构造函数）
//则默认构造函数不会被自动生成
//除非我们显式定义它

//注意：默认构造函数就是一个无参的构造函数
```

```
//只要声明了构造函数 那么默认的那个自动生成的无参构造就没用了
//相当于不会自动生成无参构造
```

```
//如果声明了其他有参构造的情况下
//之后希望使用无参构造函数去声明对象
//那需要显示的声明一个无参构造
```

使用结构体描述矩形的信息，长，宽；创建一个矩形，对其长宽进行初始化，并打印矩形的长、宽、面积、周长等信息。

使用结构体描述玩家信息，玩家名字，玩家职业
调用户输入玩家姓名，选择玩家职业，最后打印玩家的攻击信息
职业：
战士（技能：冲锋）
猎人（技能：假死）
法师（技能：奥术冲击）

打印结果：猎人唐老狮释放了假死

自定义数据类型（类型别名）

2026年10月6日 13:44

```
#pragma region 知识点一 什么是自定义数据类型
//其实我们之前学习的枚举、结构体都可以称为自定义数据类型
//但是我们这节课要学的自定义数据类型主要是指
//为原有数据类型定义一个别名
//利用
//typedef关键字 和 using关键字
//可以帮助我们为数据类型定义别名
```

```
#pragma region 知识点二 typedef的使用
//语法格式
//typedef 原变量类型 新类型名;
```

```
#pragma region 知识点三 using的使用
//语法格式
//using 新类型名 = 原变量类型;
```

```
#pragma region 知识点四 别名的作用
//代替复杂的、不方便书写和记忆的基本类型，提高高代码的可读性
//比如：用别名代表一个复杂的函数指针
//typedef 返回值 (*别名)(参数类型, 参数类型, .....);
//using 别名 = 返回值 (*)(参数类型, 参数类型, .....);
```

宏

2026年10月6日 13:53

```
#pragma region 知识点回顾 什么是编译器
//编译器是一种翻译程序
//它用于将 源语言程序 翻译为 目标语言程序
//编译器的主要任务是让人类可读的代码变成计算机能执行的指令

//源语言程序:   某种程序设计语言写成的,比如C++、C#、C、Java等语言写的程序
//目标语言程序: 机器代码或中间代码，让计算机等可以识别理解的内容
```

```
#pragma region 知识点一 什么是预处理指令
//预处理器指令 指导编译器 在实际编译开始之前对信息进行预处理
//预处理器指令 都是以#开始
//预处理器指令不是语句，所以它们不以分号;结束
//目前我们经常用到的 折叠代码块 就是预处理器指令
#pragma endregion
```

```
#pragma region 知识点二 宏的基本概念
//宏就是一种预处理指令
//它的主要作用就是在编译之前 替代源代码中的信息
//类似一连串信息的别名（信息可以是数值、字符串、表达式等等）
//宏主要通过
//#define指令来创建
//分为 不带参数的宏 和 带参数的宏

//使用注意:
//1.一般写在代码文件的最上方（#include下方）
//2.宏命名一般全为大写
//3.不需要在行末加分号;
```

```
#pragma region 知识点三 宏的使用 不带参数的宏
//基本语法
//#define 宏名 用于替代的信息
//主要作用
//定义常量或简化代码的重复部分
```

```
#pragma region 知识点四 宏的使用 带参数的宏
//基本语法
//#define 宏名(参数) 参数使用表达式
//主要作用
//简化表达式、代码重用、条件表达式、避免函数调用开销
//注意事项
//1.使用带参数的宏时要小心，因为它们不会进行类型检查
//  相对来说，内联函数更安全
//2.表达式中括号的重要性
```

括号很重要 例如

```
#define MUL(x,y) x*y
#define ADD(x,y) x+y
```

```
// 3 * 4 + 1 = 13
cout << MUL(3, 4 + 1) << endl;
// 3 * 3 + 4 = 13
cout << 3 * ADD(3, 4) << endl;
```

修改

```
#define MUL(x,y) (x*y)
#define ADD(x,y) (x+y)
```

```
#pragma region 知识点五 取消宏定义
//undef 宏
//undef表示取消宏
//主要用在宏定义符号时使用
```

```
#pragma region 知识点六 宏的使用 定义一个符号
//基本语法
//#define 符号
//主要作用
//定义一个符号，类似一个没有值的变量
//可以配合编译指令
//#ifdef 宏
//#elif defined(宏)
//#else
//#endif
//使用
//用宏来决定编译哪些代码
//可以利用它来区分不同版本执行不同的代码等功能
```

```
#pragma region 知识点七 宏的优缺点
//优点:
//1.简化代码
//2.条件编译
//3.消除函数调用的开销
//4.灵活性（没有类型限制）
//5.便于全局修改
//等等
```

```
//缺点:
//1.增加调试难度
//   宏在代码中展开，调试时可能难以追踪错误，错误信息可能不够清晰
//2.没有类型检查
//3.命名冲突
//   可能会与其他变量或函数名冲突
//4.缺乏作用域
//   宏没有作用域限制，它们在整个文件中有效
//5.可读性下降
//   过度使用宏，尤其是在复杂的宏定义中，可能会导致代码难以理解和维护
//等等

//宏虽然看起来挺有用
//但是日后再使用时
//如果能够被常量、内联函数、枚举等等功能替代宏功能时
//就不要使用宏了
```

```
#define
#define
```

排序

2026年10月6日 14:23

一、冒泡排序

```
#pragma region 知识点一 排序的基本概念
//排序是计算机内经常进行的一种操作，其目的是将一组“无序”的记录序列调整为“有序”的记录序列
//常用的排序例子
// 8 7 1 5 4 2 6 3 9
// 把上面的这个无序序列 变为 有序（升序或降序）序列的过程
// 1 2 3 4 5 6 7 8 9 （升序）
// 9 8 7 6 5 4 3 2 1 （降序）

//在程序中 序列一般 存储在数组当中
//所以 排序往往是对 数组进行排序
int arr[] = { 8, 7, 1, 5, 4, 2, 6, 3, 9, };
//把数组里面的内容变为有序的
```




```
#pragma region 知识点二 冒泡排序基本原理
// 8 7 1 5 4 2 6 3 9
// 两两相邻
// 不停比较
// 不停交换
// 比较n轮
```

```
#pragma region 知识点三 代码实现
int arr[] = { 8, 7, 1, 5, 4, 2, 6, 3, 9 };
//第一步 如何比较数组中两两相邻的数
//比较第n个和第n+1个数的大小
//我们比较的是n 和 n+1 因此 长度应该-1 避免越界
for (int n = 0; n < sizeof(arr)/sizeof(int) - 1; n++)
{
    if (arr[n] > arr[n + 1])
    {
        //第二步 满足条件的话 就去交换两个数
        //中间商不赚差价
        int temp = arr[n];
        arr[n] = arr[n + 1];
        arr[n + 1] = temp;
    }
}
//第三步 如何换m轮
//有几个数 就比较多少轮
for (int m = 0; m < sizeof(arr) / sizeof(int); m++)
{
    for (int n = 0; n < sizeof(arr) / sizeof(int) - 1; n++)
    {
        if (arr[n] > arr[n + 1])
        {
            //第二步 满足条件的话 就去交换两个数
            //中间商不赚差价
            int temp = arr[n];
            arr[n] = arr[n + 1];
            arr[n + 1] = temp;
        }
    }
}
```

```
//第四步 优化
//1.确定位置的数字 不用在比较了
//  确定了一轮后 极值（最大或者最小值）已经放到了对应的位置上了（往后放）
//  所以 没完成n轮后 后面位置的数 就没有必要再参与比较了
for (int m = 0; m < sizeof(arr) / sizeof(int); m++)
{
    for (int n = 0; n < sizeof(arr) / sizeof(int) - 1 - m; n++)
    {
        if (arr[n] > arr[n + 1])
        {
            //第二步 满足条件的话 就去交换两个数
            //中间商不赚差价
            int temp = arr[n];
            arr[n] = arr[n + 1];
            arr[n + 1] = temp;
        }
    }
}
```

```
//2.特殊情况的优化
//如果已经提前完成排序了，就应该不比了，就直接结束即可
//如果发现比较一轮下来，没有再次进行两个数的交换 那就证明已经是目标序列了
```

Int isSort = false;(声明bool变量)

二、选择排序

```
#pragma region 知识点一 选择排序基本原理
// 8 7 1 5 4 2 6 3 9
// 新建中间商
// 依次比较
// 找出极值（最大或最小）
// 放入目标位置
// 比较n轮
```

```
#pragma region 知识点二 代码实现

int arr[] = { 8,7,1,5,4,2,6,3,9 };

//先写一轮应该怎么排序
//第一步：新建中间商 来记录索引
int index = 0;
//第二步：依次比较
//每次一轮比较 从索引1开始 因为默认用于记录索引的中间商是0
//那么没有必要和自己进行比较
```

```
for (int n = 1; n < sizeof(arr)/sizeof(int); n++)
{
    //第三步：比较找出极值
    if (arr[index] < arr[n])
        index = n;//记录更大值的索引值（选择最大的数的索引）
}
//第四步：选择结束后 再放入目标位置
//数组长度 - 1 - 轮数 就是我们的目标位置
//我们在交换之前应该判断 是不是自己换自己
if (index != sizeof(arr) / sizeof(int) - 1 - 轮数)
{
    int temp = arr[index];
    arr[index] = arr[sizeof(arr) / sizeof(int) - 1 - 轮数];
    arr[sizeof(arr) / sizeof(int) - 1 - 轮数] = temp;
}
```

优化与冒泡排序一样

```
//总结
//基本概念
// 新建中间商
// 依次比较
// 找出(选择)极值
// 放入目标位置
// 比较n轮

//套路写法
//两层循环
//外层轮数 - 初始索引
//内层寻找 - 记录极值
//内层循环结束后再交换
```

写一个函数，实现一个数组的排序。可以根据参数决定是升序还是降序
利用函数指针参数实现

```
for (int m = 0; m < sizeof(arr) / sizeof(int); m++)
{
    //每一轮开始 就将标识改为false 表示没有进行交换
    isSort = false;
    for (int n = 0; n < sizeof(arr) / sizeof(int) - 1 - m; n++)
    {
        //如果for循环一轮 都没有进入两个值的交换  那就证明已经达到目的了
        if (arr[n] > arr[n + 1])
        {
            //第二步 满足条件的话 就去交换两个数
            //中间商不赚差价
            int temp = arr[n];
            arr[n] = arr[n + 1];
            arr[n + 1] = temp;
            //进行了两个值的交换
            isSort = true;
        }
    }
    //如果isSort是false 那就证明一轮结束后 没有数产生交换
    //意味着 序列已经是目标序列了 就不用再进行下一轮比较了
    //这时 就停止比较了 跳出循环
    if (!isSort)
        break;
}
```

```
//总结
//基本概念
//两两相邻
//不停比较
//不停交换
//比较m轮

//套路写法
//两层循环
//外层轮数
//内层比较
//满足交换

//如果优化
//1.比过不比
//2.加入bool
```

定义一个数组，长度为20，每个元素值随机0~100的数
使用冒泡排序进行升序排序并打印
使用冒泡排序进行降序排序并打印

写一个函数，实现一个数组的排序，并返回结果。可以根据参数决定是升序还是降序

面向对象-概念

2026年10月7日 10:00

如何学习面向对象

面向对象关键知识

类（class关键词）

面向对象三大特性

封装 + 继承 + 多态

面向对象七大原则

开闭原则、依赖倒转原则、里氏替换原则、单一职责原则

接口隔离原则、合成复用原则、迪米特法则

如何学习面向对象

重点中的重点

封装：用程序语言来形容对象。

继承：复用封装对象的代码；儿子继承父亲，复用现成代码

多态：同样行为的不同表现，儿子继承父亲的基因但是有不同的行为表现

面向对象-封装

2020年10月7日 10:08

一、类和对象

```
#pragma region 知识点一 什么是类
//基本概念
// 具有相同特征
// 具有相同行为
// 一类事物的抽象
// 类是对象的模板
// 可以通过类创建出对象
//
// 类的关键词
// class
```

```
#pragma region 知识点二 类在哪里声明和定义
//类一般
// 在.h结尾的头文件中声明
// 在.cpp结尾的源文件中定义
// 对于小型类或者模板类，可以在头文件中直接完成声明和定义
// 因此我们一般需要新建脚本文件
```

```
#pragma region 知识点五 什么是(类)对象
//基本概念
// 类的声明 和 类对象(变量)声明是两个概念
// 类的声明 类似 枚举 和 结构体的声明 类的声明相当于声明了一个自定义变量类型
// 而对象 是类创建出来的
// 相当于声明一个指定类的变量
// 类创建对象的过程 一般称为实例化对象
```

```
#pragma region 知识点六 实例化对象基本语法
//步骤一：引用类头文件

//步骤二：声明对象
//普通对象（栈上）
//类名 变量名;
//类名 变量名 = 构造函数名(...);

//指针对象（堆上）
//类名* 变量名 = new 构造函数名(...);

//引用对象（根据引用对象自己决定在栈还是堆）
//类名& 变量名 = 想要引用的对象;
```

```
//总结
//类和对象相关知识点
//和我们的结构体中的 结构体和对象 几乎一致
//类和结构体 都是我们自定义的变量类型 数据和方法的集合
//他们更像是 对象的模板 是形容(抽象)一类对象的代码
//如果要真正使用它们，我们需要进行实例化
//也就是创建对象

//需要注意的就是
//类的声明和定义可以在.h和.cpp中分开进行
//类的声明中多了一些新的知识点
//在使用别的文件中声明和定义的类时，需要先引用对应的头文件 否则无法正常使用类
```

将下面的食物进行分类，可重复

机器人、机器、人、猫、张阿姨、隔壁老王、汽车、飞机、向日葵、菊花、太阳、星星、荷花

```
GameObject* A = new GameObject();
GameObject* B = A;
B = nullptr;
A目前等于多少?
```

```
GameObject* A = new GameObject();
GameObject* B = A;
B = new GameObject();
A和B有什么关系?
```

二、成员变量和访问修饰符

```
#pragma region 知识回顾 类和对象
// 声明类
// 类一般
// 在.h结尾的头文件中声明
// 在.cpp结尾的源文件中定义
// 对于小型类或者模板类，可以在头文件中直接完成声明和定义

//class 类名
//{
    //访问修饰符:(可省略)
    //构造函数、析构函数
    //特征——成员变量
    //行为——成员方法
    //等等
//}

//声明 类对象
//步骤一：引用类头文件
```

```
//步骤二：声明对象
//普通对象（栈上）
//类名 变量名;
//类名 变量名 = 构造函数名(...);

//指针对象（堆上）
//类名* 变量名 = new 构造函数名(...);

//引用对象（根据引用对象自己决定在栈还是堆）
//类名& 变量名 = 想要引用的对象;
```

```
#pragma region 知识点一 成员变量
//基本规则
//1.声明在类语句块中
//2.用来描述对象的特征
//3.可以是任意变量类型
//4.数量不做限制
//5.是否赋值根据需求来定（老版本C++中不能直接在声明时赋值，只能在构造函数中赋值）
//6.不能有非指针类型的自己类型的成员
```

```
#pragma region 知识点二 访问修饰符
// 3P
// public — 公共的  自己(内部)和别人(外部)都能访问和使用（完全公开，任何地方都能访问）
// private — 私有的  自己(内部)才能访问和使用  不写 默认为private（严格私有，只有当前类内部才能访问）
// protected — 保护的  自己(内部)和子类才能访问和使用（对自己和子类开放，外部不可访问）
// 决定类内部的成员 的 访问权限
```

```
//一定要对类中的成员变量进行初始化
//1.在声明时 直接初始化
//2.在构造函数中初始化

//如果我们不对成员变量进行初始化
//那么他们的初始值会是一种不确定的状态
//我们一定要避免这种情况
//因此我们需要对他们进行初始化
```

```
//总结
//成员变量
//描述特征
//类中声明
//数量任意
//类型任意
//一定要初始化
//不能有非指针类型的自己类型的成员
//
//3p
//public 公共 内外
//private 私有 内
//protected 保护的 内和子类

//结构体、枚举可以在.h头文件中声明和定义
//要使用时和类一样 引用头文件即可
```

定义一个人类，有姓名，身高，年龄，家庭住址等特征

用人创建若干个对象

四、构造函数

```
#pragma region 知识点一 构造函数（结构体中讲解过的内容）
//1.基本概念：
//  构造函数是一个特殊的成员函数
//  是用来对类对象进行初始化用的
//  当我们声明一个类变量（对象）时，会自动调用构造函数

//2.基本规则：
//  2-1.没有返回值
//  2-2.函数名必须和类名相同
//  2-3.可以被重载
//  2-4.两种赋值方式
//      2-4-1:在函数中赋值
//      2-4-2:利用成员初始化列表来初始化（更高效）
//  2-5.若传入参数和成员变量重名，可以使用 this->成员变量 来表示是成员变量
```

```
//如果声明成员变量时也初始化了
//那么以 构造函数中初始化的值为准
//只有当没有构造函数时 才会直接应用 声明时初始化的内容
```

```
//3.调用构造函数的几种方式
//  3-1.栈上分配-不带new
//      3-1-1.类型 类对象(参数...)
//      3-1-2.类型 类对象{参数...}
//      3-1-3.类型 类对象 = 类型(参数...);
//      3-1-4.类型 类对象 = 类型{参数...};
//      3-1-5.类型 类对象 = {参数...};
//  3-2.堆上分配-带new
//      类型* 类对象 = new 类型(参数);

//4.类还有两个默认的构造函数
//  4-1.传入 const 左值引用的构造（拷贝构造）
//      主要作用：避免不必要的复制，允许使用已有的对象用来初始化成员
//      一般用来处理复制另一个对象的数据
//  4-2.传入 右值引用的构造（移动构造）
//      主要作用：接收将要被移动的临时对象或其他右值，可以提高性能
```

```
//5.如果我们没有为结构体定义任何构造函数
//  编译器会生成一个默认的构造函数
//  如果定义了其他构造函数（如带参数的构造函数）
//  则默认构造函数不会被自动生成
//  除非我们显式定义它
//  注意：默认构造函数就是一个无参的构造函数
```

```
#pragma region 知识点二 补充知识点 委托构造函数
//所谓的委托构造函数
//就是允许一个构造函数调用另一个构造函数，以减少代码重复
//基本语法：
//构造函数():另一个重载的构造函数(参数)
//{}
//它的主要作用，说人话：
//在调用某一个构造函数之前可以通过这种方式先调用另一个构造函数
//达到复用相同代码的作用，这样我们在一些构造函数中就可以少些一些代码逻辑了
```

基于成员方法练习题

对人类的构造函数进行重载，用人类创建若干个对象

基于成员变量练习题

对班级类的构造函数进行重载，用班级类创建若干个对象

写一个Ticket类，有一个距离变量（在构造对象时赋值，不能为负数），有一个价格特征，有一个方法GetPrice可以读取到价格，并且根据距离distance计算价格price（1元/公里）

0~100公里 不打折

101~200公里 大9.5折

201~300公里 打9折

300公里以上 打8折

有一个显示方法，可以显示这张票的信息。

例如：100公里100块钱

五、析构函数

```
#pragma region 知识点 析构函数（结构体中讲解过的内容）
//1.基本概念
//  析构函数 是当对象被释放时会调用的方法
//  也就意味着在析构函数中主要处理的就是堆内存回收相关的工作
//  因为堆内存空间需要我们自己手动回收
//  它的主要作用就是用来当类对象被释放时
//  能够将其中在堆上分配的内容清理掉，避免内存泄漏

//2.基本规则
//  声明定义在类中
//  2-1.没有返回值,没有参数
//  2-2.函数名必须和类名相同，并且在前面加~符号
//  2-3.只能有一个析构函数，不能重载
//  2-4.如果类中没有定义析构函数，编译器会自动生成一个默认的析构函数，该函数会依次调用所有成员的析构函数
//      注意：默认析构函数不会释放动态资源（即堆上new的成员）

//3.析构函数不需要我们手动调用
//  它是自动调用的函数
//  当对象的生命周期结束时，析构函数会自动被调用
//  对象释放时机：
//  3-1.栈中的内容，当对象超出作用域时就会释放
//  3-2.堆中的内容，delete 删除对象时就会释放
```

—唐老狮—

```
//总结
//析构函数 虽然不需要我们手动调用 但是它的存在是非常重要的
//对象的生命周期结束时会自动的去调用
//栈：语句块结束后 对象会自动释放 这时就会调用析构函数
//堆：手动delete时 就会去调用析构函数

//我们就只需要把重点放在析构函数中 去释放类或结构体中声明在堆上分配内存的成员
```

```
class A {
private:
    int* ptr;
public:
    A() { ptr = new int(100); }
    ~A() { }
};

int main() {
    A obj;
    return 0;
}
```

该代码是否存在内存泄漏？为什么？

如果存在应该怎么修改代码？

```
class A {
public:
        B bObj;
        A() { }
        ~A() { }
};

class B {
public:
        A aObj;
        B() { }
        ~B() { }
};
int main() {
    B obj;
    return 0;
}
```

此段代码是否存在问题？具体是什么问题？

六、静态成员

```
#pragma region 知识回顾 static变量
//静态变量
```

七、 const成员

```
#pragma region 知识回顾 常量
//常量是指在程序执行过程中其值不能被改变的
//关键点：
//1.必须初始化
//2.不能被修改
//指向常量的指针：  const在 *前面去添加即可
//指针常量：const在 *后面去添加即可，值可
//指向常量的指针常量：const在 *前后都加，
```

```
//第二种：构造函数中初始化，只能通过初始化
Lesson7::Lesson7(int testI):testI(testI
{
    //不能这样初始化 常量const成员
    //this->testI = testI;
}
```

```
#pragma region 知识点二 const成员函数
//规则
//在声明完成员函数后 在后方加上const关键字
//作用
//1.函数中只能读取成员变量，不能修改成员变
//2.const对象只能调用const方法，不能调用非
//对于类对象来说 它本质上是一个自定义的变量
//它也是可以作为一个常量
```

```
#pragma region 知识点三 mutable关键字
//如果我们想要在const方法中修改成员变量
//我们可以在声明该变量时，在前面加上mutab
```

```
//总结
//const 成员变量
//它和常量的特点是一样的，用来修饰指针规则
//需要注意初始化方式
//1.直接在声明时定义
//2.通过构造函数传值定义，但是需要使用初始

//const 成员方法
//1.函数如果用const修饰，那么在其中就不能
//2.对于const类对象、只能访问其中的const

//mutable关键字
//让普通成员能够在const方法中被修改
```

我们有几种初始化类中const成员变量的方法？

```
class Counter {
private:
  int count;
public:
  Counter() : count(0) {}

  void increment() { count++; }

  int getCount() {
    return count;
  }

  int getConstCount() const {
    return count;
  }
};

int main() {
  const Counter c;
  //1.这行代码是否能编译？为什么？如果不能，如何修
  cout << c.getConstCount() << endl;
  //2.这行代码能否编译？为什么？如果不能，如何修改
  cout << c.getCount() << endl; //
  return 0;
}
```

八、const修饰函参

```
#pragma region 知识点一 const修饰函数参
//  规则：返回值 函数名(const int等基础
//  作用：不允许传入的参数在内部修改（但是
```

```
#pragma region 知识点二 const修饰函数参数
//  1.指向能改，值不能改（相当于指向常量的
//  规则：返回值 函数名(const 类型* 参数名
//  作用：该指针可以修改指向，但是指向的值
//
//  2.指向不能改，值能改（相当于指针常量）
//  规则：返回值 函数名( 类型* const 参数
//  作用：该指针可以修改指向，但是指向的值
//
//  3.指向不能改，值也不能能改（相当于指向
//  规则：返回值 函数名( const 类型* cons
//  作用：该指针可以修改指向，但是指向的值
//
//  一般推荐使用第一种方式，可以提高代码安
```

```
#pragma region 知识点三 const修饰函数参数
//  规则：返回值 函数名( const 类型& 参数
//  作用：常量对象和非常量对象都可以传入，
```

为什么要用const修饰函数参数？请说出你的理解

九、友元函数

十、友元类

十一、友元成员函数

变量，大部分的常量在声明时都必须被初始化

，值不能改，指向可以改
以改，指向不能改
值和指向都不能改了

列表的形式初始化 常量const成员
t)

量
Econst方法
类类型声明的对象

e关键字

也和外部一样

化列表的方式进行初始化 不能再里面去写赋值

去修改成员变量了,只能去获取
方法，普通成员无法访问

改?
?

数：基本类型
型 参数名)
意义不大，因为就算内部修改也不会影响外部)

：指针
指针)
名)
不能修改，主要用于放置修改指针指向内容的值

名)
不能修改，主要用于放置修改指针指向内容的值

常量的指针常量)
t 参数名)
不能修改，主要用于放置修改指针指向内容的值

全性

：引用
名)
但不能修改对象本身，由于是引用，还可以避免拷贝开销

十二、嵌套类

十三、命名空间

```
#pragma region 知识点一 命名空间是什么
//命名空间是为了防止命名冲突而设计的一种机制
//我们在声明全局变量、函数、类等等内容时，是不能够在同一作用域中重名的
//而有了命名空间，我们可以在不同命名空间中声明同名内容
//
//命名空间就像是一个工具包，在其中可以放置一些相同名字的工具
```

```
#pragma region 知识点二 命名空间的声明和定义
//在哪里声明和定义：
//命名空间在.h和.cpp中都可以声明和定义
//1.如果在.cpp中声明和定义就只能在该文件中使用，无法在其他文件中使用
//2.如果在.h中声明，那么可以通过引用头文件的形式，在其他文件中使用
//  定义可以直接在.h中也可以分开在.cpp中，建议在.h中声明，在.cpp中定义
//  注意：如果在.h中定义全局变量，需要加上extern关键字
//  (extern关键字在此处的作用是 在命名空间中写全局变量时，
//   如果变量要在多个文件中共享使用，就必须在头文件中写 extern 声明，在 .cpp 文件中写实际定义
```

```
//声明语法：
/*namespace 命名空间名
{
    变量
    函数
    类
    等等
}*/

//注意：
//1..h中声明 全局变量需要加上 extern
//2..h中声明 .cpp中实现 那么如果是命名空间中的类中的内容需要定义 需要通过 命名空间名::类名::方法定义
```

也可以在命名空间中定义 需要在类之外通过 类名::方法定义

```
#pragma region 知识点三 命名空间的使用
//想要使用命名空间，除了引用头文件外
//1.需要指定使用内容来自哪个命名空间
//  基本语法：
//  命名空间名::变量名（函数名或类名等等）


//2.或者可以在顶端使用 using namespace 命名空间名;
//  这样可以在使用时省略前面的内容
//  注意：需要注意使用不同命名空间时，其中有同名内容问题
#pragma region 知识点四 命名空间可以分散式声明
//如果我们在多个文件中声明同一个命名空间
//那么他们代表的是一个命名空间
#pragma region 知识点五 嵌套命名空间
//命名空间可以类似嵌套类一样无限嵌套声明
#pragma region 知识点六 命名空间别名
//基本语法：
//namespace 自定义名 = 命名空间名
//作用：
//可以简化复杂的命名空间名
#pragma region 知识点七 匿名命名空间
//匿名命名空间以变量、函数、类的作用域只在当前 .cpp 文件中
//防止和其他文件冲突，实现更好的封装，是 static 的现代替代品
//只在当前文件中可见，允许在多个.cpp文件中定义同名函数或变量时，不会有重定义问题

//注意：不能在头文件中使用匿名命名空间
```

请简单总结命名空间声明定义时应该注意些什么?

```
namespace A {
    int x = 5;
}

namespace B {
    int x = 10;
}

using namespace A;
using namespace B;

int main() {
    std::cout << x << std::endl;
}
```

上述代码能否通过编译？如果不能，哪里错了？如何修改？

十四、运算符重载

```
//protected 保护的 内和子类

//结构体、枚举可以在.h头文件中声明和定义
//要使用时和类一样 引用头文件即可
```

定义一个人类，有姓名，身高，年龄，家庭住址等特征
用人创建若干个对象

定义一个学生类，有姓名，学号，年龄，同桌等特征。用学生类创建若干个学生

定义一个班级类，有专业名称，教室容量，学生等
创建一个班级对象

```
Person p;
p.age = 10;
Person p2;
p2.age = 20;
```
请问p.age为多少?

```
Person p
p.age = 10;
Person p2 = p;
p2.age = 20;
```
请问p.age为多少?

```
Student s;
s.age = 10;
int age = s.age;
age = 20;
```
请问s.age为多少?

```
Student s;
s.deskmate = new Student();
s.deskmate.age = 10;
Student* s2 = s.deskmate;
s2.age = 20;
```
请问s.deskmate.age为多少?

三、成员方法 以及 防止头文件重复编译

```
#pragma region 知识点一 成员方法的声明和定义
//基本概念
//成员方法(函数) 用来表现对象行为
//1.声明在类语句块中
//2.是用来描述对象的行为的
//3.规则和函数声明规则相同
//4.受到访问修饰符规则影响
//5.返回值参数不做限制
//6.方法数量不做限制
//7.定义位置有几种选择
//  方式一:
//  在类内声明并定义 (小型函数、极短的构造函数)

//  方式二:
//  在.h内声明，在.cpp中定义 (推荐使用该方法)

//  虽然c++中允许以上两种写法，但是都建议大家使用方式二（之后学习的内联函数和模板类除外）
```

```
#pragma region 知识点二 成员方法的使用
//成员方法 必须实例化出对象 再通过对象来使用 相当于该对象执行了某个行为
//栈上


//堆上
```

```
#pragma region 知识点三 内联成员方法
//在类中将成员方法变成内联函数的作用和外部一样

//内联函数回顾:
//使用 inline 关键字对函数进行声明或定义
//主要作用是告诉编译器，在调用该函数时，将函数的代码直接插入到调用该函数的位置
//而不是执行常规的函数调用过程，
//这样做的目的是减少函数调用的开销
//适用于哪些小型的、频繁调用的函数
```

```
//注意:
//直接在类内定义的成员方法，默认是内联的
//也就是我们在知识点一中讲解的方式一定义函数的方式
//当我们在类外定义成员方式时，可以显示的加上inline关键词让其成为内联函数
//但是在使用时需要特别注意:
//1.是否真正内联是由编译器决定的，inline只是建议
//2.内联函数不能过大，否则可能导致代码膨胀
//3.内联函数不适用于递归函数，否则可能导致无限展开
//4.虚函数不能是内联

//因此对于内联函数知识点来说，主要做了解即可
//大多数时候，完全可以不用显示使用内联，影响不会太大
```

```
#pragma region 知识点四 头文件中的防止重复编译命令
//什么是头文件重复编译
//头文件可能会被多个源文件（.cpp）或同一个源文件间接多次包含
//如果果没有适当的防护机制，会导致编译错误(一般会报重复定义的错误)

//C++中为了防止重复编译
//提供以了2种方式

//1.使用传统宏定义来防止头文件重复编译
//  #ifndef 自定义宏名  // 如果没有定义 自定义宏名
//  #define 自定义宏名  // 定义 自定义宏名

//  #endif  // 结束条件编译

//2.使用更加方便的pragma指令来防止重复编译
//  #pragma once
```

基于成员变量练习题
为人类定义说话、走路、吃饭等方法

基于成员变量练习题
为学生类定义学习、吃饭等方法

定义一个食物类，有名称，热量等特征
思考如何和人类以及学生类联系起来

基于成员变量练习题
为人类一个私有成员变量money（表示拥有多少钱）
想办法让其能在外部可以被获取，但是不能被修改

如何防止头文件重复包含

```
}
```

此段代码是否存在问题？具体是什么问题？

六、静态成员

```
#pragma region 知识回顾 static变量
//静态变量
//用法:
// static 变量类型 变量名;
//作用:
// static修饰的变量会在程序的整个生命周期内存储
// 就算在函数内部声明，也遵循这一规则
// 它可以用来声明静态 局部变量、静态全局变量 或 静态类成员 等
// 声明一次，永久使用
//生命周期:
// 静态局部变量的生命周期是整个程序运行期，即使函数返回，它也不会被销毁。
// 静态全局变量的生命周期也是整个程序运行期

//优点:
// 静态变量只初始化一次，且一直保留值，适合全局共享信息或缓存的场景
//缺点:
// 静态变量会一直占用内存空间，直到程序结束，因此会增加内存消耗
//注意:
// 除非有需求，否则不要过多的使用静态变量

//静态全局变量和全局变量的区别
//他们的主要区别就是作用域不同
```

```
//全局变量:
//可以在多个文件中访问，适合需要共享数据的情况
//静态全局变量:
//只能在定义它的文件中访问，适合只在单个源文件内部使用的数据
```

```
#pragma region 知识点一 静态成员基本概念
//类成员也可以用静态关键词static进行修饰
//用static修饰的 成员变量、方法
//称为静态成员
```

```
#pragma region 知识点二 静态成员的特点
//共同特点:
// 1.在声明成员 (变量和函数) 时，前面加上static关键字
// 2.直接用类名::出使用，也可以通过类对象访问 (不推荐)
// 3.生命周期贯穿程序始终 (类似全局变量)
// 4.属于整个类。而不是某个对象，所有对象共享同一个静态成员

//静态成员变量特点:
//在.h中声明后，需要在.cpp中进行定义

//静态成员函数特点:
//1.静态成员函数中不能使用非静态成员
//2.非静态成员函数可以使用静态成员
//3.没有this指针
```

```
//在静态成员方法中无法使用非静态成员变量或者方法
//因为非静态成员变量和方法 是基于对象的 (对象实例化才会有的内容)
//而静态成员是基于类的 (是程序一开始就会分配内存的)
```

```
#pragma region 知识点三 为什么可以不依赖对象使用静态成员
//记住!
//程序中是不能无中生有的
//我们要使用的对象、变量，函数都是要在内存中分配内存空间的
//之所以要实例化对象，目的就是分配内存空间，在程序中产生一个抽象的对象

//静态成员的特点
//程序开始运行时 就会分配内存空间，存储在数据段 (数据存储区)，所以我们就能直接使用。
//静态成员和程序同生共死
//只要使用了它，直到程序结束时内存空间才会被释放
//所以一个静态成员就会有自己唯一的一个"内存小房间"
//这让静态成员就有了唯一性
//在任何地方使用都是用的小房间里的内容，改变了它也是改变小房间里的内容。
```

```
//对于成员函数和静态成员函数来说 他们都是存储在代码段的
//只是通过static来判断 哪些成员函数是通过对象点出来用 哪些静态成员函数是通过类名::来使用
//函数内部是否可以使用this或者成员变量也是通过static来判断的
```

```
#pragma region 知识点四 静态成员对于我们的作用
//静态变量:
//1.常用唯一变量的声明
//2.方便别人获取的对象声明
//3.节约一定的内存空间
//静态方法:
//常用的唯一的方法声明 比如 相同规则的数学计算相关函数
```

```
//总结
//                普通成员变量                                静态成员变量
//存储区别      堆、栈、和对象同生共死                    数据段、和程序同生共死
//使用区别      需要通过对象点出                          通过类名::出 (也可以通过对象点出，但不建议)
//唯一性区别    每个对象都是独有的特征                    共用的唯一的特征或行为的体现 (一般用于存储数学或物理公式
//
//                普通成员方法                                静态成员方法
//存储区别                          都在代码段
//函数内部逻辑区别      是可以使用普通或静态成员的                  只能使用静态成员
//函数使用区别          需要通过对象点出                            通过类名::出 (也可以通过对象点出，但不建议)
//多个对象中的节约性    如果使用静态成员方法 可以把一些共用的逻辑提取处理 作为唯一的一个函数
//                      这样就不需要再多个类中重复书写相同代码了 想要跨类使用 直接 通过类名::来使用即可
```

一唐老狮一

请简单描述
静态成员和普通成员的区别

调用静态成员相关知识实现
一个类对象，在整个应用程序的生命周期中，有且仅会有一个该对象的存在，不
能在外部实例化，直接通过该类类名就能够得到唯一的对象

面向对象-继承

2026年10月8日 19:01

一、基本规则

```
#pragma region 知识点一 基本概念
//一个类A继承一个类B
//类A将会继承类B的成员（不包括构造函数、析构函数和友元等）
//A类将拥有B类的特征和行为（根据访问权限决定能否访问）

//被继承的类
//称为 父类、基类、超类

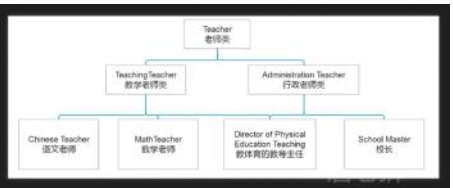
//继承的类
//称为子类、派生类

//子类可以有自己的特征和行为

//特点
//1.传递性 子类可以间接继承父类的父类（前提是访问权限允许）
//2.多继承 子类可以有多个父类
```

```
#pragma region 知识点二 基本语法
//class 子类名 : 继承方式 父类名
//{
//    //子类自己的成员
//};

//继承方式就是使用访问修饰符来
//改变父类成员在子类中的访问方式
//
//回顾访问修饰符作用
// 3P
// public — 公共的  自己(内部)和别人(外部)都能访问和使用（完全公开，任何地方都能访问）
// private — 私有的  自己(内部)才能访问和使用  不写 默认为private（严格私有，只有当前类内部才能访问）
// protected — 保护的  自己(内部)和子类才能访问和使用（对自己和子类开放，外部不可访问）
// 决定类内部的成员 的 访问权限
//
//父类成员原本访问级别    public继承后        protected继承后        private继承后
//     public              public              protected              不可访问
//     protected           protected           protected              不可访问
//     private             不可访问            不可访问               不可访问
```




```
#pragma region 知识点四 子类和父类的同名成员
//C++中允许子类存在和父类同名的成员
//但是不建议这样使用

//子类同名成员会覆盖父类成员
//但可以通过作用域解析符::来强制访问父类成员
//比如:  父类名::成员

//子类同名不同参数成员函数会隐藏父类成员
//当子类定义了一个和父类同名但参数不同的函数
//父类中所有同名函数都会被隐藏，即使参数不同也不能再直接访问!
//可以通过 using 父类名::父类方法名 来解决隐藏问题
```

```
#pragma region 知识点五 强行禁止类被继承
//如果我们不希望一个类可以被继承
//我们可以通过final关键词来修饰类
//基本语法:
//class 类名 final
//{
//}
//在类名后加上final关键字，就可以禁止该类被继承了
```

二、继承后的构造、析构函数

三、多重继承

四、菱形继承

五、继承中的友元和运算符重载

六、里氏替换原则

```
#pragma region 知识点一 基本概念
// 里氏替换原则是面向对象七大原则中最重要的原则
// 概念:
// 任何父类出现的地方，子类都可以替代
// 重点:
// 语法表现—父类容器装子类对象,因为子类对象包含了父类的所有内容
// 作用:
// 方便进行对象存储和管理
```

```
#pragma region 知识点二 基本实现
//栈上分配
GameObject player = Player();
GameObject monster = Monster();
GameObject boss = Boss();

GameObject objects[3] = { Player(), Monster(), Boss() };

//堆上分配
GameObject* player2 = new Player();
GameObject* monster2 = new Monster();
GameObject* boss2 = new Boss();

GameObject* objects[3] = { new Player(), new Monster(), new Boss() };
```

```
#pragma region 知识点三 栈上的里氏替换
//虽然我们可以在栈上用父类容器装载子类对象
//但是不建议大家使用栈上的里氏替换
//主要原因:
//1.对象切片：用父类容器装载子类对象，子类部分会被裁掉
//2.无法转换：无法将父类容器转换为子类对象

//因此，绝大多数情况下，我们应当避免在栈上使用里氏替换
```

```
#pragma region 知识点四 堆上的里氏替换
//当在堆上使用里氏替换原则
//用父类容器装载子类对象
//不会出现对象切片，并且可以进行转换
//我们可以通过 dynamic_cast 将父类容器转为子类对象使用
//当学了多态后，更能体会这一点（因为需要虚函数）
```

```
//一个转换的方法
//<>是传入想要转换为的类型
//()是具体想要转换谁
```

面向对象-多态

2026年10月8日 19:01

一、虚函数

```
#pragma region 知识点一 多态的概念
// 多态按字面的意思就是"多种形态"
// 让继承同一父类的子类们 在执行相同方法时有不同的表现（状态）
// 主要目的
// 同一父类的对象 执行相同行为（方法）有不同的表现
// 解决的问题
// 让同一个对象有唯一行为的特征
```

```
#pragma region 知识点二 解决的问题
Father* f = new Son();
f->SpeakName();
Son* s = dynamic_cast<Son*>(f);
s->SpeakName();
```

```
#pragma region 知识点三 多态的实现
//我们目前已经学过的多态
//编译时多态—函数重载，运算符重载，一开始就写好的

//我们将学习的:
//运行时多态
//实现运行时多态的关键词是:
//v:virtual（虚函数）
//o:override（推翻重写）
//v和o一定是结合使用的 来实现多态
```

```
//栈上分配的这种对象 存在对象切片时 就不会执行子类中重写的方法 所以一般我们不会这样使用
GameObject player2 = Player("唐老狮2");
player2.Atk();

//栈上分配，但是用引用父类容器装载，是不会对象切片的 是可以执行子类中重写的方法的
Player p2 = Player("唐老狮3");
GameObject& g = p2;
g.Atk();
```

```
#pragma region 知识点四 如何保留父类逻辑
//1.如果想要在重写函数中保留父类逻辑
//  只需要通过 父类名::方法名 显示调用父类逻辑即可
//2.如果想要在外部调用，同理
player->GameObject::Atk();
```

```
void Atk() override
{
    GameObject::Atk();
    cout << "玩家" << name << "进行攻击" << endl;
}
```

```
#pragma region 知识点五 override可以省略
//override是可以省略的
//但是极其不建议省略
//因为省略可能由于拼写错误或者结构错误而出现问题
```

```
//总结
//多态：让同一类型的对象，执行相同行为时有不同的表现
//解决的问题：让同一对象有唯一的行为特征
//vo:
//v:virtual 虚函数
//o:override 推翻重写
//v和o一定是配合使用的 来实现我们的多态
//但是想要保留父类逻辑的话，我们可以显示的去调用父类方法即可 父类名::方法名
//override是可以省略的，但是极其不建议大家省略 因为可能出现不必要的问题
```

```
真的鸭子嘎嘎叫，木头鸭子吱吱叫，橡皮鸭子唧唧叫

所有员工9点打卡
但经理十一点打卡，程序员不打卡

创建一个图形类，有求面积和周长两个方法
创建矩形类，正方形类，圆形类继承图形类
实例化矩形、正方形、圆形对象求面积和周长
```

二、虚析构函数

```
#pragma region 知识点一 虚析构函数的作用
//在 C++ 的多态中，虚析构函数是极其重要的概念
//用于确保通过父类指针删除子类对象时，能正确调用子类的析构函数，避免资源泄漏或行为未定义

//说人话:
//如果用父类指针装载子类指针对象
//在对父类指针容器进行释放时，如果父类析构函数不是虚函数
//会导致无法执行子类析构函数，造成内存泄漏等问题
```

```
//注意点:
//1.如果存在多层继承关系，那么我们只需要将最上层的析构函数设置为 虚函数即可
//2.多重继承，只需要保证最上层单的基类 析构函数时虚函数即可
//  A     B
//   \   /
//     C
//  A和B都需要将析构函数设置为虚函数
//     A
//   /   \
//  B     C
//   \   /
//     D
//  A的析构函数设置为虚函数
```

栈上分配可不用

```
//总结
//当存在继承关系时，都建议大家使用虚析构函数
//因为我们无法保证不会使用指针（堆上）的里氏替换来释放对象
```

三、抽象类

```
#pragma region 知识点一 抽象类
//概念:
//抽象类是指包含至少一个纯虚函数的类
```

四、禁止重写

```
#pragma region 知识回顾 强行禁止类被继承
//如果我们不希望一个类可以被继承
//我们可以通过final关键词来修饰类
//基本语法:
//class 类名 final
//{
//}
//在类名后加上final关键字，就可以禁止该类被继承了
```

```
#pragma region 知识点 禁止重写
//final关键字除了可以禁止类被继承
//还可以限制函数不能再被重写
//语法规则:
//在函数尾部加上 final
//它只能搭配 virtual和override使用
//因为它是用来禁止虚函数被重写的
```

```
//如果在虚函数后面直接加上final
//意义是
//1.让子类无法重写
//2.让子类也无法去写一个同名函数
virtual void Test() final;
```

五、虚函数表

```
#pragma region 知识回顾
//在程序运行时，系统中会有不同的内存存储区域
//
//我们目前学习过的知识，涉及的存储区域有:
//1.代码段（Code Segment，或称代码存储区）：
//  存储 函数的机器码（执行指令）等信息的区域
//  通常是只读的，防止程序意外修改自己的代码
//  在程序加载时就被加载到内存中
//  生命周期较长，与程序同生共死
//
//2.数据段（Data Segment，或称数据存储区）：存储 全局变量、静态变量 等信息的区域
//  一般分成两个子区域
//  初始化数据段：存储已初始化的全局和静态变量
//  未初始化数据段（BSS Segment）：存储未初始化的全局和静态变量，通常初始化为零
//  生命周期较长，与程序同生共死
//
//3.常量段（Constant Segment，或称常量存储区）：存储 字符串字面量和其他只读数据 信息的区域
//  通常是只读的，以防止程序修改常量的值
//  比如 cout << "Hello, World" << endl 中的 "Hello, World" 就存储在常量区
//  生命周期由程序控制
//  注意：我们声明的常量变量不存储在常量段，而是根据在哪里声明决定存储在哪里
```

```
//4.栈（Stack）：存储 局部变量、函数参数、返回值 等信息的区域
//  由编译器自动管理，随着函数的调用和返回而分配和释放
//  生命周期与函数调用相同，快速分配和释放
//
//5.堆（Heap）：存储 动态分配的内存数据 等信息的区域
//  由程序员手动管理，需要主动分配，主动释放
//  生命周期不固定，由程序员决定生死
```

```
#pragma region 知识点一 虚函数表的基本概念
//虚函数表（简称：V-Table）是C++中多态实现的底层原理
//编译器会为每一个有虚函数的类创建至少一个虚函数表
//虚函数表是一个数组，表中存储的是该类虚函数的函数指针地址
//虚函数表被类的所有对象共享（所有实例化的某类对象都共用该类的虚函数表）
//虚函数表通常是存储在数据段或常量段（更常见）的
//不管实例化的对象在堆、栈、静态等哪个存储区，对应类的虚函数表永远只有一份
//每个对象中隐藏了至少一个虚函数表指针（vptr）
//该指针在对象构造时自动赋值，指向对应类的虚函数表
```

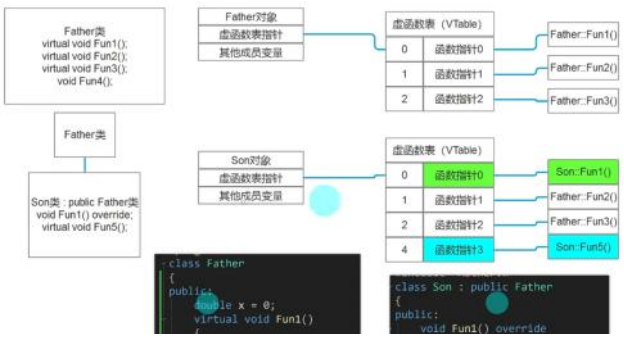
```
#pragma region 知识点二 虚函数表的工作原理
//类定义时:
//编译器为每个有虚函数的类生成一张虚函数表，存储在数据段或常量段（更常见）
//虚函数表的本质是一个指针数组信息
//指针数组中每个元素存储着虚函数的函数指针（指向该类对应虚函数的代码实现）

//实例化对象时:
//存在虚函数的对象内部都会至少有一个隐藏的虚函数表指针（简称：V-Ptr）
//该指针指向虚函数表

//调用虚函数时:
//运行时会通过VPtr指向的VTable查表调用相应的函数

//注意:
//1.只有有虚函数的类才会有虚函数表
//2.由于存在虚函数的类对象中存在隐藏的指针
//  因此sizeof时得到的字节数会至少多一个指针(64位8字节,32位4字节)的大小
```

举例



```
//因为我们无法保证不会使用指针（基类）的里氏替换来释放对象
```

三、抽象类

```
#pragma region 知识点一 抽象类
//概念：
//抽象类是指包含至少一个纯虚函数的类
//基本特点：
//1.不能直接实例化
//2.只能被继承
//3.若子类继承抽象类后不实现纯虚函数，也会变成抽象类
//4.抽象类中可以有成员变量和普通函数
```

```
#pragma region 知识点二 纯虚函数
//纯虚函数就是没有被实现的虚函数
//语法：
//virtual 返回类型 函数名(参数列表) = 0;

//纯虚函数一般用来表示抽象的行为
//比如：
//    图形基类
//   /   |    \
//圆   正方形   长方形
```

例如

```
class Fruit
{
private:
    string name;
public:
    Fruit(string name)
    {
        this->name = name;
    }

    virtual ~Fruit()
    {

    }

    //纯虚函数 因为不同的水果被吃的方式不同
    virtual void Eat() = 0;

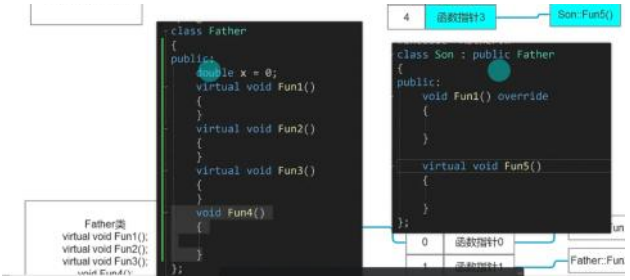
    void Washing()
    {
        cout << "洗水果:" << name << endl;
    }
};
```

```
#pragma region 知识点三 举例说明
Fruit* a = new Apple("苹果");
a->Washing();
a->Eat();
delete a;
a = nullptr;

Fruit* b = new Banana("香蕉");
b->Washing();
b->Eat();
delete b;
b = nullptr;
```

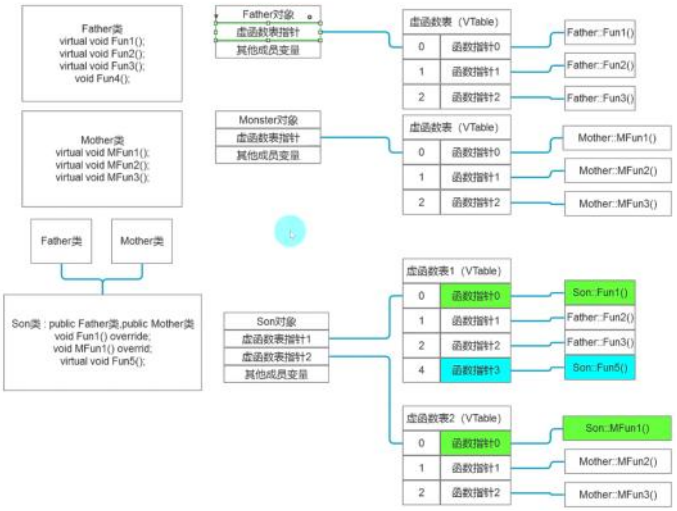
写一个动物抽象类，写三个子类
人叫，狗叫，猫叫

创建一个图形类，有求面积和周长两个方法
创建矩形类，正方形类，圆形类继承图形类
实例化矩形、正方形、圆形对象求面积和周长




```
#pragma region 知识点三 多继承和单继承的区别
//单继承中，子类的虚函数表可以理解为是复制父类的虚函数表后再修改
//比如修改其中的指向，添加新的自己的虚函数

//而在多重继承中，有多少个父类就会有多少张虚函数表
//声明子类对象时，对象中会有多个虚函数表指针指向对象的虚函数表
//对于子类自己的虚函数，将会放入第一个虚函数表中（主流编辑器的实现方式）
```



面向对象-关联知识

2026年10月10日 13:13

一、接口

二、多继承时的同名虚函数

三、空类型所占字节数

四、字符数组和string

```
#pragma region 知识回顾 字符数组
//字符数组就是类型为字符char的一维数组

//常用方式:
//方式一: 聚合初始化
char charArr[5] = { 'H', 'E', 'L', 'L', 'O' };
char charArr2[] = { 'H', 'E', 'L', 'L', 'O' };

//方式二: 单独初始化
char charArr3[5];
charArr3[0] = 'H';
charArr3[1] = 'E';

//这几种声明初始化的方式, 如果直接将字符数组拿来打印 会出现问题

//可以直接使用字符串对字符数组进行赋值
char charArr4[] = "Hellow World"; //string
cout << charArr4 << endl;
//原本看起来只有12个字符的字符串 但是长度却有13个
int length = sizeof(charArr4) / sizeof(char);
cout << length << endl;
```

```
//通过观察长度, 我们发现了一个潜在规则
//对字符数组用字符串去赋值时, 会默认在尾部加上一个\0的转移字符
//代表字符串的结束

char charArr5[5];
charArr5[0] = 'A';
charArr5[1] = 'B';
//charArr5[2] = 'X';
//charArr5[3] = 'C';
//charArr5[4] = '\0';
//如果不对字符数组后面的内容进行初始化 并且其中没有\0
//在打印时就不知道何时结束, 就会不停的往后读取
//因为没有明确的终止符, 程序会根据内存中是否存在 '\0' 停止打印
```

```
//可能打印出多余字符, 甚至是乱码
//也就是说可能出现访问到字符数组范围之外的内存区域, 产生未定义的行为
cout << charArr5[2] << endl;
cout << charArr5 << endl;
//关于"烫"字符
//在内存中, 汉字字符使用多字节编码 (比如 UTF-8 或 GBK)
//当程序读取到这些内存内容时, 会尝试解释它们。
//由于编码规则, 有可能形成合法的汉字编码, 导致控制台上显示出汉字或其他非预期字符
```

```
#pragma region 知识点一 字符数组公共方法
//C++中内置一些关于字符数组的方法
//方便我们对其进行操作
//注意:
// 这些方法其实都是C语言的方法 大家了解即可
//1.strcat_s(字符数组, 字符数组长度, 字符串)
//  作用: 将一个字符串连接到另一个字符数组中
//        如果字符数组长度不够会报错
char str1[10] = "123";
char str2[10] = "456";
strcat_s(str1, sizeof(str1), str2);
cout << str1 << endl;
```

```
//2.strcpy_s(字符数组, 字符数组长度, 字符串)
//  作用: 将一个字符串复制到另一个字符数组中
//        如果字符数组长度不够会报错
strcpy_s(str1, sizeof(str1), "7777777");
cout << str1 << endl;

//3.strcmp (字符数组1, 字符数组2)
//  作用: 比较两个字符数组是否相同
//  0 代表相同
//  不是0 就是不相同
cout << strcmp(str1, "7777777") << endl;
```

```
//4.strlen (字符数组)
//  作用: 得到字符数组实际长度
cout << strlen(str1) << endl;
```

```
#pragma region 知识点二 string相关方法
//1.拼接 (C++入门中学习过)
//  通过 + 或者 append方法进行拼接
string string1 = "123";
string string2 = "456";
string1.append(string2);
cout << string1 << endl;

//2.返回字符串长度
//  size()/length()
cout << string1.size() << endl;
cout << string1.length() << endl;
```

```cpp
//3.判断字符串是否为空
//  empty()
string string3 = "1";
cout << string3.empty() << endl;

//4.清空字符串
//  clear()
string3.clear();
cout << string3 << endl;

//5.返回容量
//  capacity()
string3 = "1234567890123456789";
cout << string3.capacity() << endl;
```

```cpp
//6.在指定位置插入字符串
//  insert(位置索引, 字符串)
//  注意: 不要超出最大位置 (包含\0考虑)
string3.insert(1, "哈哈");
cout << string3 << endl;
string3.insert(23, "嘻嘻嘻");
cout << string3 << endl;
```

```cpp
//7.从指定索引位置起删除固定个长度个字符
//  erase(位置索引, 长度)
//  注意: 不要超出最大位置 (包含\0考虑)
cout << string3.size() << endl;
string3.erase(29, 6);
cout << string3 << endl;

//8.替换指定区间为另一个字符串
//  replace(开始位置索引, 长度, 字符串)
string3.replace(0, 10, "123456");
cout << string3 << endl;
```

```cpp
//9.交换两个字符串
//  swap(另一个字符串)
cout << string1 << endl;
cout << string2 << endl;
string1.swap(string2);
cout << string1 << endl;
cout << string2 << endl;

//10.删除最后一个字符
//  pop_back();
cout << string3 << endl;
string3.pop_back();
cout << string3 << endl;
```

注意:

```cpp
//10.删除最后一个字节 需要注意 破坏多个字节字符的问题
//  pop_back();
string3.push_back('1');
cout << string3 << endl;
string3.pop_back();
cout << string3 << endl;

//11.添加单个字符
//  push_back(字符)
string3.push_back('A');
cout << string3 << endl;
```

```cpp
//12.查找字符串第一次出现的位置
//   find(字符或字符串)
string string4 = "12345627";
cout << string4.find("2") << endl;

//13.查找最后一次出现的位置(从尾部找)
//   rfind(字符或字符串)
cout << string4.rfind("2") << endl;
```

可用数值去存储这个位置，如果找没有的字符则该数值为0

```cpp
//14.查找首次/最后一次 出现传入字符串中 任一字符的位置
//   首次: find_first_of(字符串)
//   最后一次: find_last_of(字符串)
string4 = "12345627";
i = string4.find_first_of("478");
cout << i << endl;
i = string4.find_last_of("478");
cout << i << endl;
```

```cpp
//15.查找第一个/最后一个 不在传入字符串中的字符
//   第一个: find_first_not_of(字符串)
//   最后一个: find_last_not_of(字符串)
i = string4.find_first_not_of("12367");
cout << i << endl;//3
i = string4.find_last_not_of("2367");
cout << i << endl;//4
```

```cpp
//16.截取字符串
//   substr(起始位置,长度)
string4 = "12345627";
string string5 = string4.substr(1, 4);
cout << string5 << endl;

//17.字符串转字符数组
//  data()
const char* charPtr = string4.data();
for (int i = 0; i < string4.size(); i++)
{
    cout << charPtr[i] << endl;
}
```

```cpp
//18.判断字符串内容是否相等
//   ==/!=
string1 = "123";
string2 = "123";
if (string1 == string2)
    cout << "相等" << endl;
```

```cpp
#pragma region 知识点三 string的本质
//stirng的本质就是一个C++中封装的一个管理字符数组的类
//它封装了动态内存分配、长度管理、字符串操作函数等复杂逻辑
//因此在现代C++开发当中，都建议大家使用string来替代字符数组
//因为这样更高效，安全
```

请写出string中提供的截取和替换对应的函数名

请将字符串1|2|3|4|5|6|7
变为　　2|3|4|5|6|7|8
并输出

编写一个函数，将输入的字符串反转。不要使用中间商，你必须原地修改输入数组。交换过程中不使用额外空间
比如：输入{ 'h' , 'e' , 'l' , 'l' , 'o' }　　输出
{ 'o' , 'l' , 'l' , 'e' , 'h' }